24CSEN1011

OBJECT ORIENTED PROGRAMMING

MODULE 4 INTERFACES AND PACKAGES

- Multiple and nested inheritance.
- **Interface:** Creating and Implementing an Interface, Using Interface References, Implementing Multiple Interfaces, Extending Interfaces, Nested Interface.
- **Packages:** Package Fundamentals,Packages in Java, Member Access, Importing Packages, Static import.
- **File I/O**

Multiple Inheritance

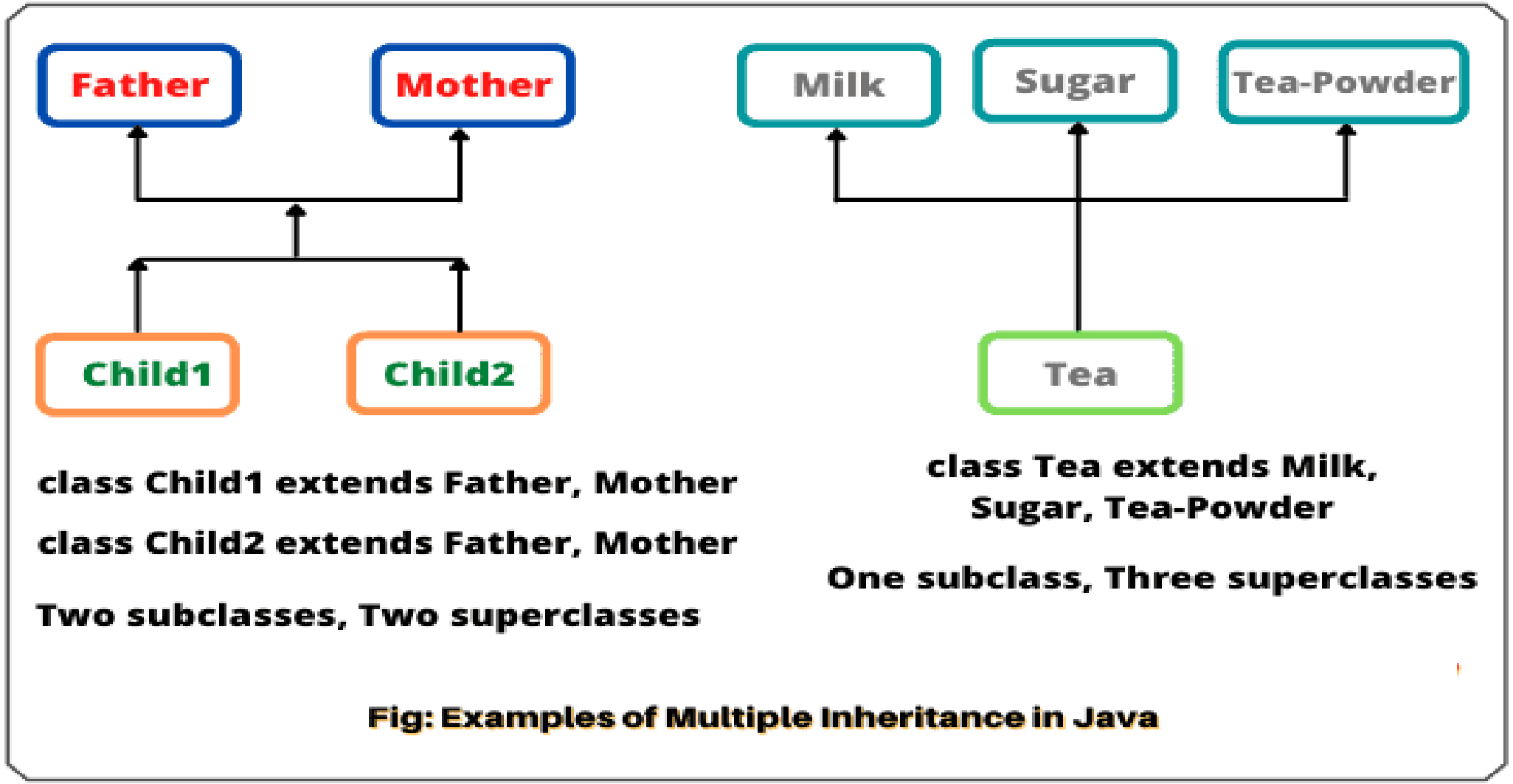


Fig: Examples of Multiple Inheritance in Java

Example 1: Multiple Inheritance Example (Father, Mother, Child)

```
interface Father {
   void showFatherTraits();
}

interface Mother {
   void showMotherTraits();
}

class Child1 implements Father, Mother {
   public void showFatherTraits() {
      System.out.println("Child1 has Father's qualities.");
   }

   public void showMotherTraits() {
      System.out.println("Child1 has Mother's qualities.");
   }
}
class Child2 implements Father, Mother {
   public void showFatherTraits() {
      System.out.println("Child2 has Father's qualities.");
   }
   public void showMotherTraits() {
      System.out.println("Child2 has Mother's qualities.");
   }
}

public class MultipleInheritanceExample1 {
   public static void main(String[] args) {
      Child1 c1 = new Child1();
      c1.showFatherTraits();
      c1.showMotherTraits();


      Child2 c2 = new Child2();
      c2.showFatherTraits();
      c2.showMotherTraits();
   }
}
```

Output:

```
Child1 has Father's qualities.
Child1 has Mother's qualities.

Child2 has Father's qualities.
Child2 has Mother's qualities.
```

Example 2: Multiple Inheritance Example (Tea, Milk, Sugar, TeaPowder)

```
interface Milk {
   void addMilk();
}

interface Sugar {
   void addSugar();
}

interface TeaPowder {
   void addTeaPowder();
}

class Tea implements Milk, Sugar, TeaPowder {
   public void addMilk() {
      System.out.println("Adding milk...");
   }

   public void addSugar() {
      System.out.println("Adding sugar...");
   }

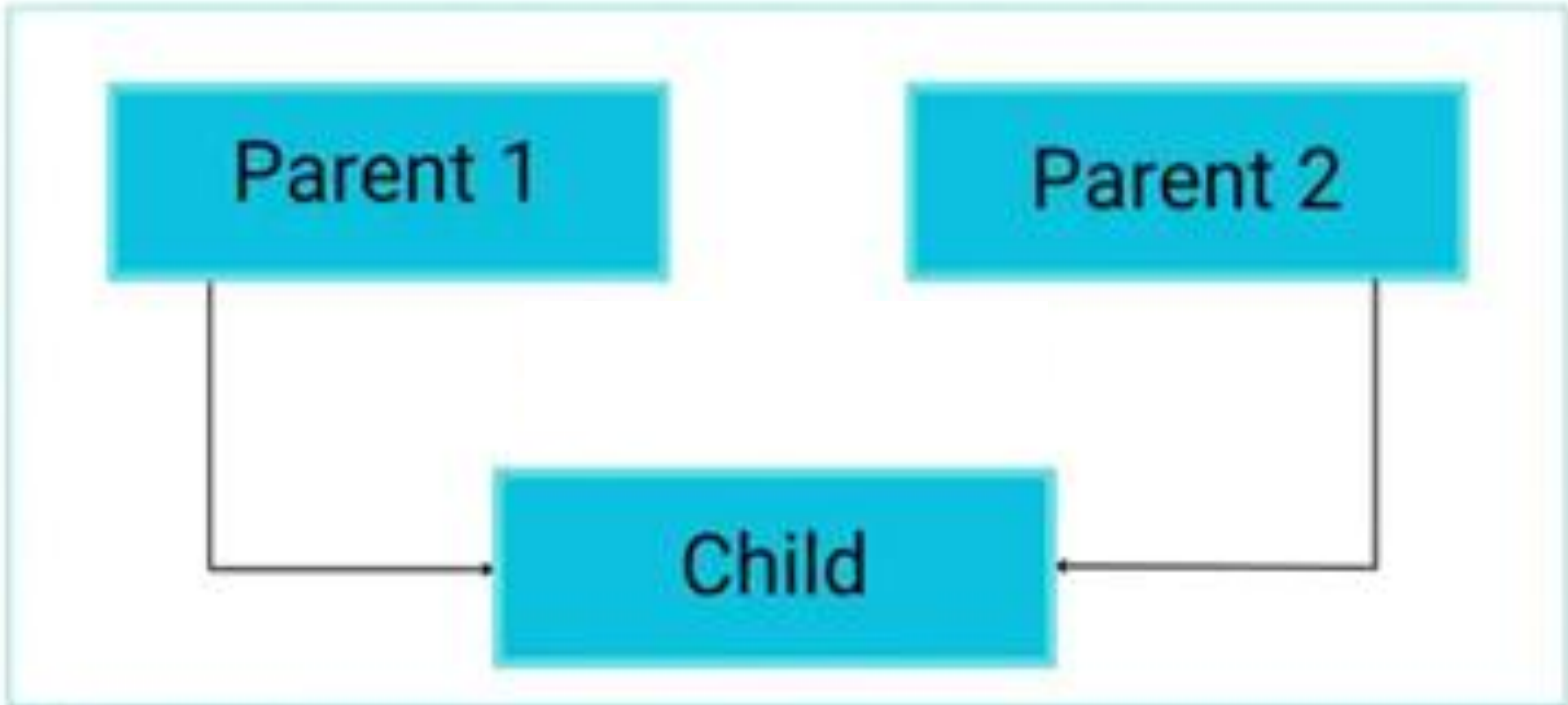
   public void addTeaPowder() {
      System.out.println("Adding tea powder...");
   }

   void prepareTea() {
      addMilk();
      addSugar();
      addTeaPowder();
      System.out.println("Tea is ready!");
   }
}

public class MultipleInheritanceExample2 {
   public static void main(String[] args) {
      Tea tea = new Tea();
      tea.prepareTea();
   }
}
```

Output:

```
Adding milk...
Adding sugar...
Adding tea powder...
Tea is ready!
```


Parent 1
Parent 2
Child

Example: Multiple inheritance using interfaces

```
interface Parent1 {
   void displayParent1();
}

interface Parent2 {
   void displayParent2();
}

// Child class implementing both interfaces
class Child implements Parent1, Parent2 {
   public void displayParent1() {
      System.out.println("This is Parent 1.");
   }

   public void displayParent2() {
      System.out.println("This is Parent 2.");
   }
   void displayChild() {
      System.out.println("This is the Child class inheriting from
Parent1 and Parent2.");
   }
}

public class MultipleInheritanceDemo {
   public static void main(String[] args) {
      Child c = new Child();
      c.displayParent1();
      c.displayParent2();
      c.displayChild();
   }
}
```

Output:

```
This is Parent 1.
This is Parent 2.
This is the Child class inheriting from Parent1 and Parent2.
```

Nested Inheritance (Multilevel Inheritance)

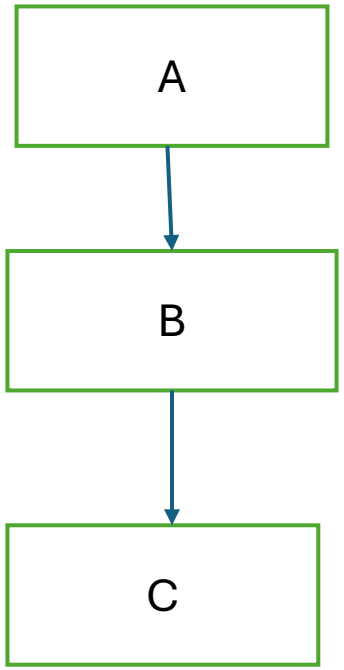
- means when a class inherits another class, and that class itself is inherited by another class — forming a **chain of inheritance**.
- It is also called **Multilevel Inheritance** — where one class serves as a parent for another derived class.

Syntax

```
class A {
    // Parent class
}

class B extends A {
    // Child class of A
}

class C extends B {
    // Child class of B (also indirectly inherits from A)
}
```



Example :Nested Inheritance

```
// Base class
class Grandfather {
    void showGrandfather() {
        System.out.println("This is the Grandfather class");
    }
}

// Intermediate class
class Father extends Grandfather {
    void showFather() {
        System.out.println("This is the Father class");
    }
}

// Derived class
class Son extends Father {
    void showSon() {
        System.out.println("This is the Son class");
    }
}
```

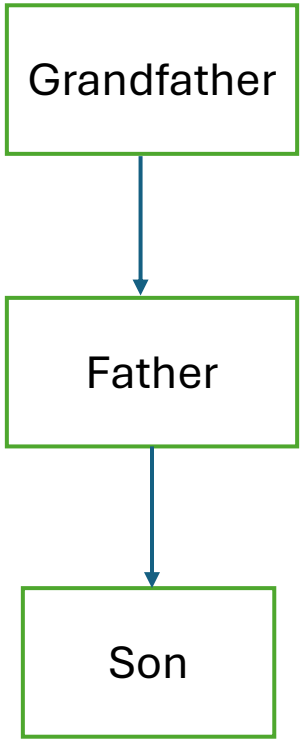
```
public class Mainclass{

    public static void main(String[] args) {
        Son obj = new Son();

        // Accessing all class methods
        obj.showGrandfather();
        obj.showFather();
        obj.showSon();
    }
}
```

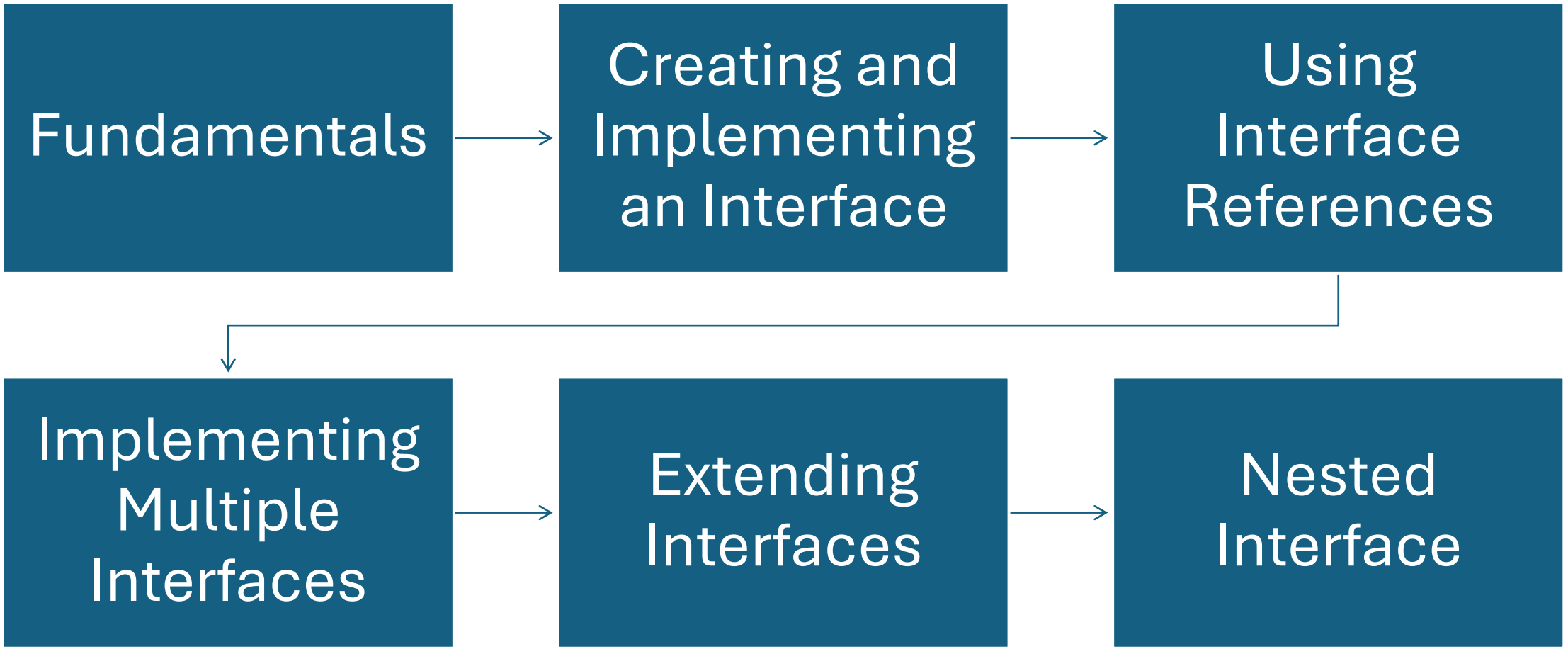
Output:

```
This is the Grandfather class
This is the Father class
This is the Son class
```



Interface

Contents



Advantages

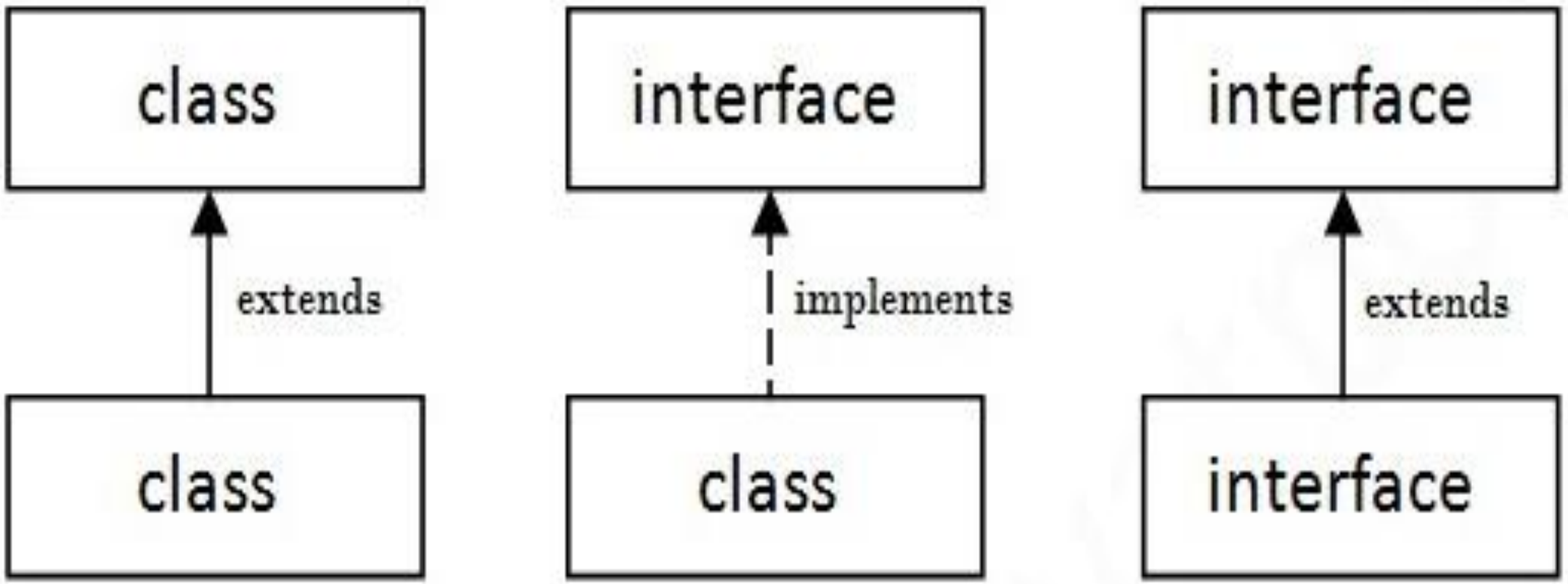


It is used to achieve complete abstraction.



It supports multiple inheritance.

Relationship between classes and interfaces



Interface

- Interface contain:**constant attributes** (public static final **variables**),**abstract methods** (methods without body **or** no definition)

Syntax:

```
interface InterfaceName {
    // constants
    int VALUE = 10;
    // abstract method
    void display();
}
```

Key points about interface constants:

1.Automatically public, static, and final(Even if you don't write these keywords, the compiler treats them as such)

```
int VALUE = 10;     // same as public static final int VALUE = 10;
```

2.Must be initialized

You **cannot declare a constant without a value**.

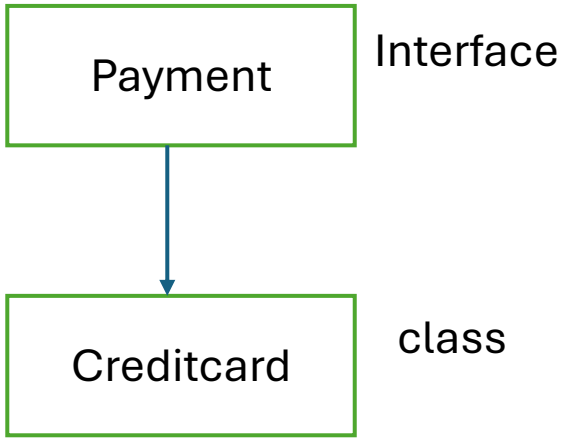
int VALUE; // ❌ Not allowed

int VALUE = 10; // ✅ Allowed

Example:Payment System

```
interface Payment {
   double CHARGE = 2.5;  // attribute (constant)
   void pay();
}

class CreditCard implements Payment {
   public void pay() {
      System.out.println("Payment done using Credit Card with charge: " + CHARGE + "%");
   }
 }
 public class Mainclass{
   public static void main(String[] args) {
      CreditCard c = new CreditCard();
      c.pay();
   }
}
```



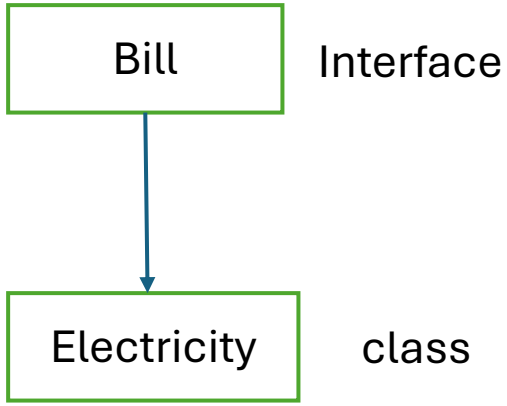
Output:

Payment done using Credit Card with charge: 2.5%

Example:Electricity Bill Calculation

```
interface Bill {
   double RATE = 7.5;                    // attribute
   void calculate(int units);
}

class Electricity implements Bill {
   public void calculate(int units) {
      double amount = units * RATE;
      System.out.println("Electricity Bill for " + units + " units = Rs" + amount);
   }
 }
 public class Mainclass{
   public static void main(String[] args) {
      Electricity e = new Electricity();
      e.calculate(100);
   }
}
```



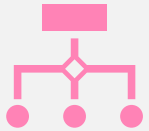
Output:

Electricity Bill for 100 units = ₹750.0

Implementing an interface (contd..)



If a class implements more than one interface, the interfaces are separated with a comma.



The methods that implement an interface must be declared public.



The type signature of the implementing method must match exactly the type signature specified in the interface definition.

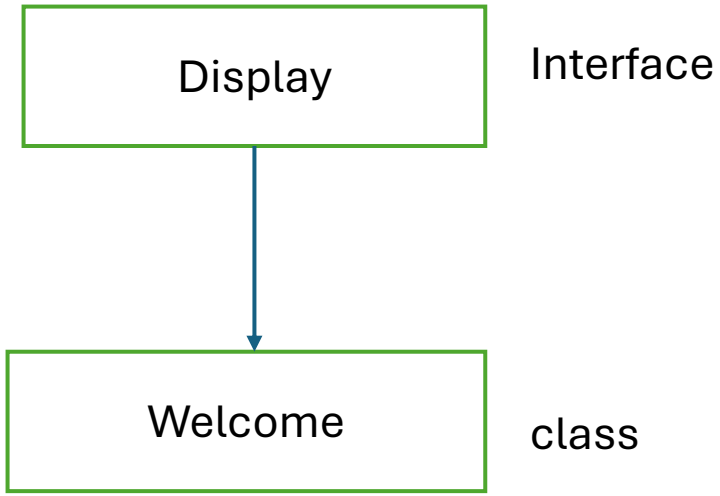
Creating and Implementing an Interface

To use an interface, a class must **implement** it and provide code for all abstract methods.

Example:Display Message

```
interface Display {
   String MSG = "Hello Java!";   // attribute
   void message();
}

class Welcome implements Display {
   public void message() {
      System.out.println("Message: " + MSG);
   }
}
public class Mainclass{
   public static void main(String[] args) {
      Welcome w = new Welcome();
      w.message();
   }
}
```



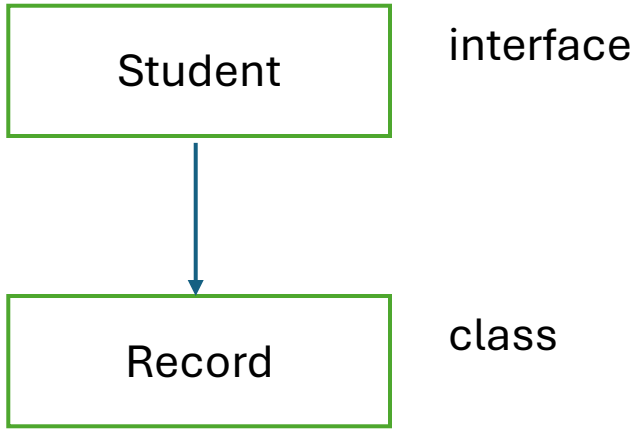
Output:

Message: Hello Java!

Example – Student Information

```
interface Student {
   String COLLEGE = "ABC College"; // attribute
   void displayDetails();
}

class Record implements Student {
   public void displayDetails() {
      System.out.println("Student belongs to: " + COLLEGE);
   }
 }
 public class Mainclass{
   public static void main(String[] args) {
      Record r = new Record();
      r.displayDetails();
   }
}
```



Output:

Student belongs to: ABC College

Using Interface References

An **interface reference** can refer to an object of any class that implements the interface. Through this, we achieve **runtime polymorphism**.

Syntax:

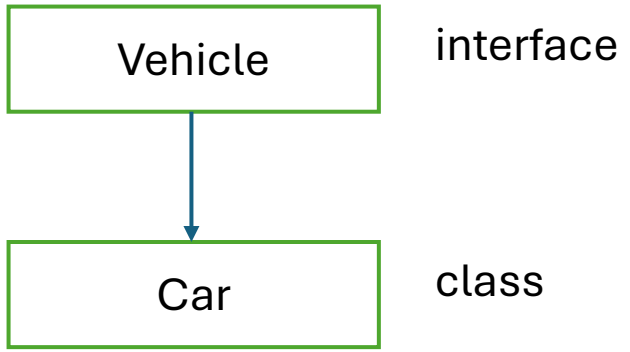
```
InterfaceName ref = new ImplementingClass();
```

Example

```
interface Vehicle {
    int WHEELS = 4;     // attribute
    void start();
}
class Car implements Vehicle {
    public void start() {
        System.out.println("Car starts with " +
WHEELS + " wheels.");
    }
}
```

```
public class Mainclass{
public static void main(String[] args) {
    Vehicle v = new Car();    // interface reference
    v.start();
}
}
```

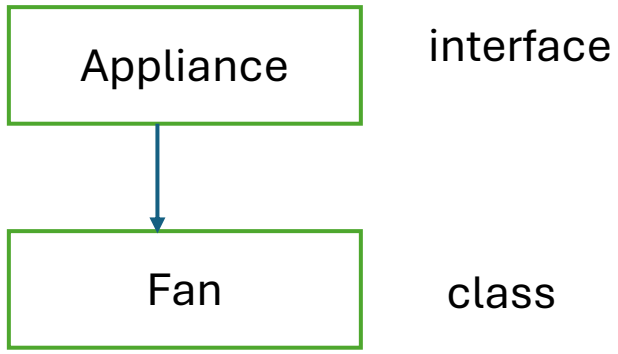
Output:Car starts with 4 wheels.



Example – Appliance Control

```
interface Appliance {
   String BRAND = "Samsung"; // attribute
   void turnOn();
}

class Fan implements Appliance {
   public void turnOn() {
      System.out.println(BRAND + " Fan is now ON.");
   }
 }
public class Mainclass{
   public static void main(String[] args) {
      Appliance a = new Fan();      // interface reference
      a.turnOn();
   }
}
```



Output:

Samsung Fan is now ON

Interfaces in Java

VEHICLE

interface

implements

class



Bus

class



Car

class



Truck

```java
// Interface definition
interface Vehicle {
    void start();          // only one abstract method
}

// Bus class implementing Vehicle interface
class Bus implements Vehicle {
    public void start() {
        System.out.println("Bus starts with diesel engine.");
    }
}

// Car class implementing Vehicle interface
class Car implements Vehicle {
    public void start() {
        System.out.println("Car starts with a key or button.");
    }
}
// Truck class implementing Vehicle interface
class Truck implements Vehicle {
    public void start() {
        System.out.println("Truck starts with heavy engine.");
    }
}
```

```java
// Main class
public class InterfaceExample {
    public static void main(String[] args) {
  Vehicle v1 = new Bus();    // Interface reference to Bus object
        v1.start();

        Vehicle v2 = new Car();    // Interface reference to Car object
        v2.start();

        Vehicle v3 = new Truck();  // Interface reference to Truck object
        v3.start();
    }
}
```

Output:

```
Bus starts with diesel engine.
Car starts with a key or button.
Truck starts with heavy engine.
```

Note:

if multiple classes derived from an interface then **interface reference** is used.

interfacename obj =new **classname();**

Implementing Multiple Interfaces

- A single class can **implement multiple interfaces** by separating them with commas.
- This helps to achieve **multiple inheritance** in Java.

Syntax:

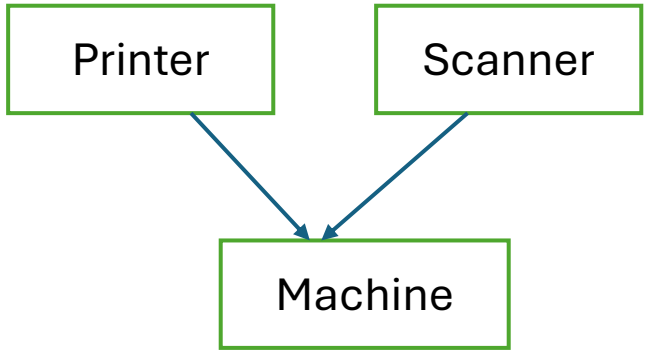
```
interface A { int X = 10; void show(); }
interface B { int Y = 20; void display(); }

class C implements A, B {
   public void show() { }
   public void display() { }
}
```

Example:Printer & Scanner

```
interface Printer {
   String BRAND = "HP";   // attribute
   void print();
}
interface Scanner {
   String TYPE = "Laser"; // attribute
   void scan();
}
```

```
class Machine implements Printer, Scanner {
   public void print() {
      System.out.println("Printing with " + BRAND + "
printer.");
   }
   public void scan() {
      System.out.println("Scanning using " + TYPE + "
scanner.");
   }
}
public class Mainclass{
   public static void main(String[] args) {
      Machine m = new Machine();
      m.print();
      m.scan();
   }
}
```



Output:

```
Printing with HP printer.
Scanning using Laser scanner.
```

Example – Smart Device (Music + Navigation)

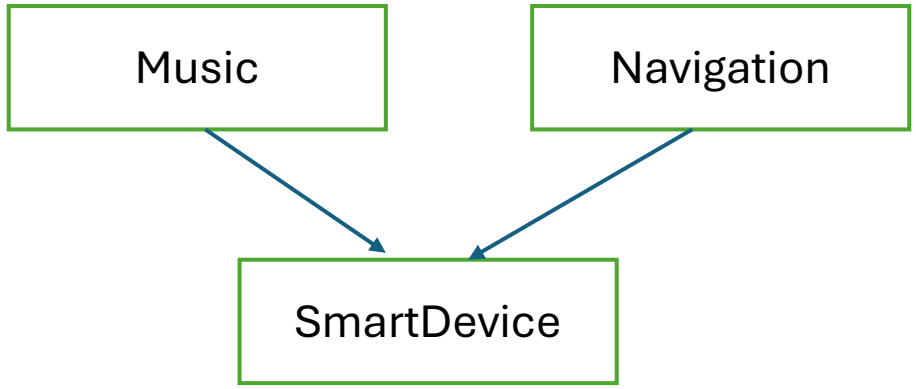
```
interface Music {
   String APP = "Spotify";        // attribute
   void play();
}
interface Navigation {
   String MODE = "GPS";           // attribute
   void map();
}

class SmartDevice implements Music, Navigation {
   public void play() {
      System.out.println("Playing music from " + APP);
   }
   public void map() {
      System.out.println("Navigating using " + MODE);
   }
}
```

```
public class Mainclass{
   public static void main(String[] args) {
      SmartDevice sd = new SmartDevice();
      sd.play();
      sd.map();
   }
}
```

Output:

Playing music from Spotify
Navigating using GPS



Extending Interfaces

An interface can **extend another interface**, inheriting its constants and abstract methods.

Syntax:

```
interface A { int VALUE = 100; void show(); }
interface B extends A { void display(); }
```

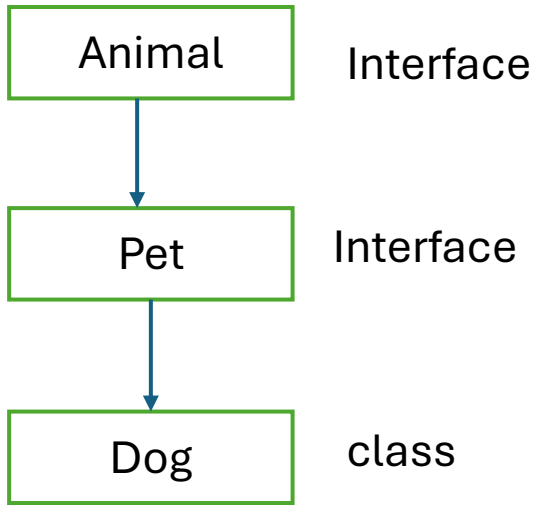
Example:Animal Behavior

```
interface Animal {
   String TYPE = "Domestic";   // attribute
   void eat();
}
interface Pet extends Animal {
   String SOUND = "Bark";      // attribute
   void play();
}
```

```
class Dog implements Pet {
   public void eat() {
      System.out.println(TYPE + " dog eats bones.");
   }
   public void play() {
      System.out.println("Dog makes sound: " + SOUND);
   }
}
public class Mainclass{
   public static void main(String[] args) {
      Dog d = new Dog();
      d.eat();
      d.play();
   }
}
```

Output:

```
Domestic dog eats bones.
Dog makes sound: Bark
```



Example – Library Management

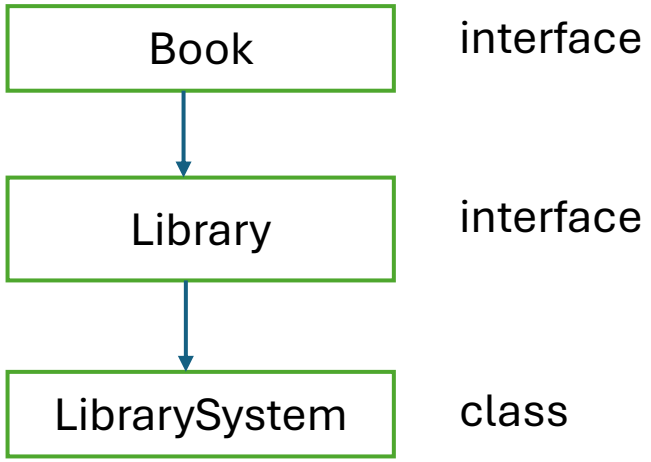
```
interface Book {
   String TYPE = "Educational"; // attribute
   void borrow();
}
interface Library extends Book {
   int MAX_DAYS = 14;          // attribute
   void returnBook();
}

class LibrarySystem implements Library {
   public void borrow() {
      System.out.println("Borrowed a " + TYPE + " book.");
   }
   public void returnBook() {
      System.out.println("Return book within " + MAX_DAYS +
" days.");
   }
}
```

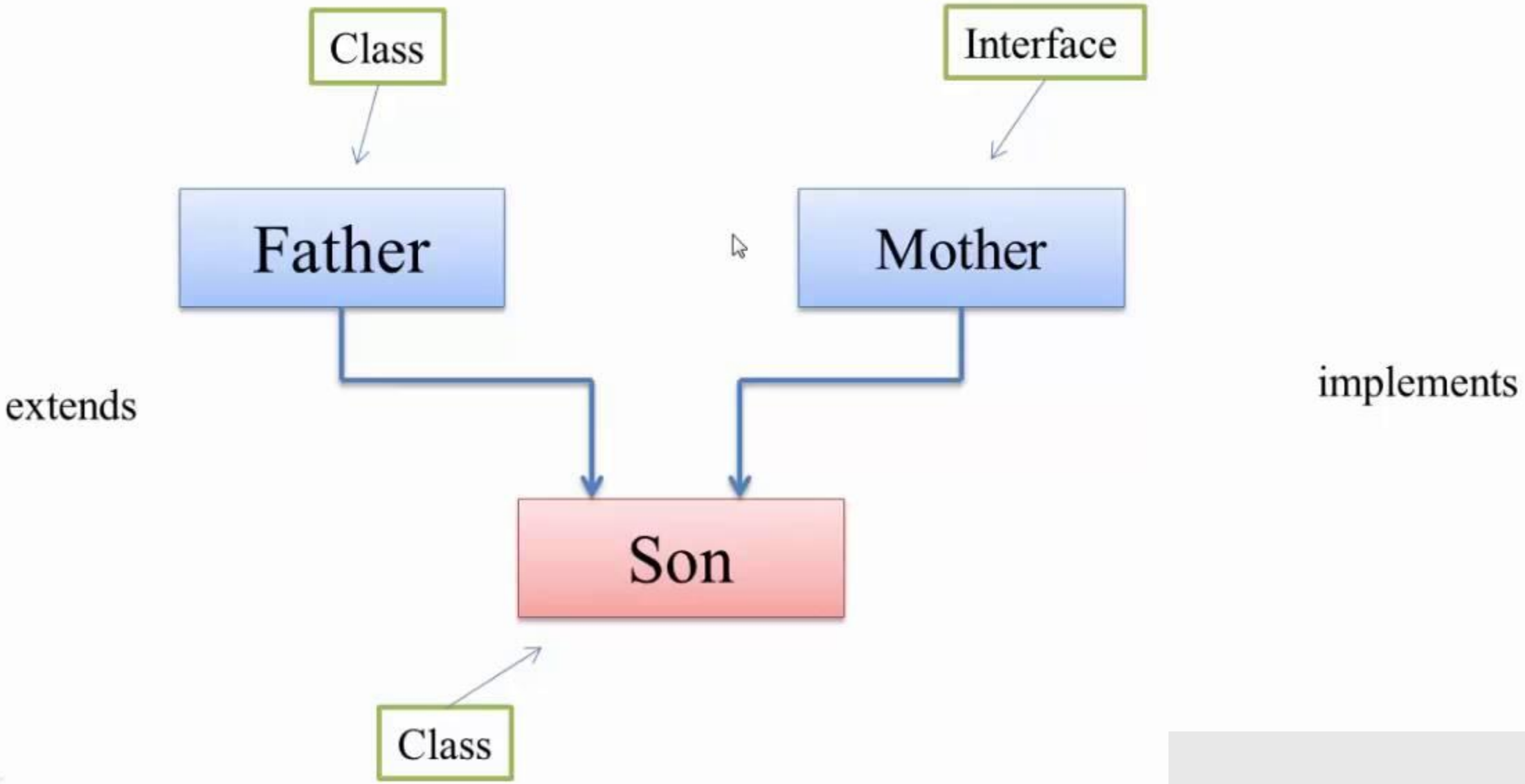
```
public class Mainclass{
   public static void main(String[] args) {
      LibrarySystem l = new LibrarySystem();
      l.borrow();
      l.returnBook();
   }
}
```

Output:

Borrowed a Educational book.
Return book within 14 days.



Extends and Implements together



```
// Parent class
class Father {
    void showFather() {
        System.out.println("This is the Father class.");
    }
}

// Interface
interface Mother {
    void showMother();
}

// Child class that extends Father and implements Mother
class Son extends Father implements Mother {
    public void showMother() {
        System.out.println("This is the Mother interface
implemented in Son class.");
    }

    void showSon() {
        System.out.println("This is the Son class.");
    }
}
```

```
public class Mainclass{
    public static void main(String[] args) {
        Son obj = new Son();
        obj.showFather();   // From Father class
        obj.showMother();   // From Mother interface
        obj.showSon();      // From Son class
    }
}
```

Output:

```
This is the Father class.
This is the Mother interface implemented in Son class.
This is the Son class.
```

Nested Interface

A **nested interface** is an interface declared inside another **interface or class**.

It is used to logically group related interfaces.

Syntax:

```
interface Outer {
   int OUTER_ID = 1;
   interface Inner {
      int INNER_ID = 2;
      void show();
   }
}
```

Example:Outer and Inner Communication

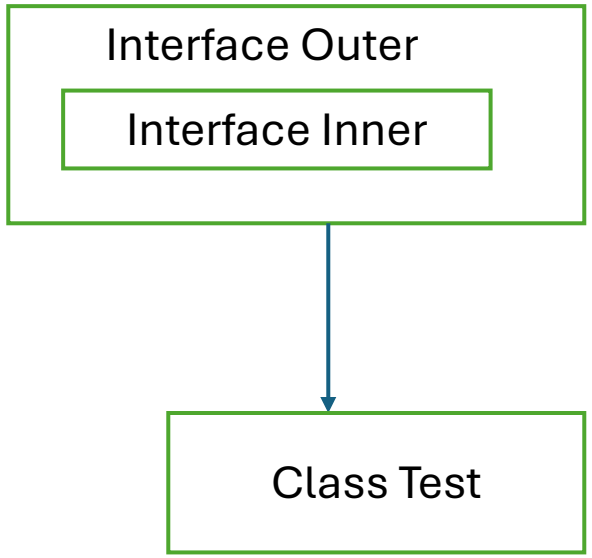
```
interface Outer {
   int OUTER_ID = 10;

   interface Inner {
      int INNER_ID = 20;
      void show();
   }
}
```

```
class Test implements Outer.Inner {
   public void show() {
      System.out.println("Outer ID: " + OUTER_ID);
      System.out.println("Inner ID: " + INNER_ID);
   }
}
public class Mainclass{
   public static void main(String[] args) {
      Outer.Inner obj = new Test();
      obj.show();
   }
}
```

Output:

Outer ID: 10
Inner ID: 20



Note:

class implements with **Outer.Inner**

To access, use **Outer.Inner** format.

Syntax

```
class OuterClass {
   interface InnerInterface {
      void display();
   }
}
```

Example

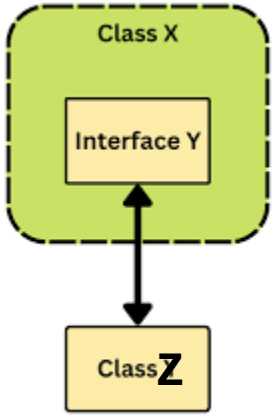
```
// Outer class
class X {
   // Nested interface inside class X
   interface Y {
      void show();
   }
}

// Class implementing the nested interface
class Z implements X.Y {
   public void show() {
      System.out.println("Nested Interface Example");
   }
}

// Main class
class NestedInterfaceExample {
   public static void main(String[] args) {
      X.Y obj = new Z();   // Interface reference using nested interface
      obj.show();          // Calls method from class Z
   }
}
```

Output:

Nested Interface Example



Example – Banking System

```
interface Bank {
   String BANK_NAME = "City Bank"; // attribute

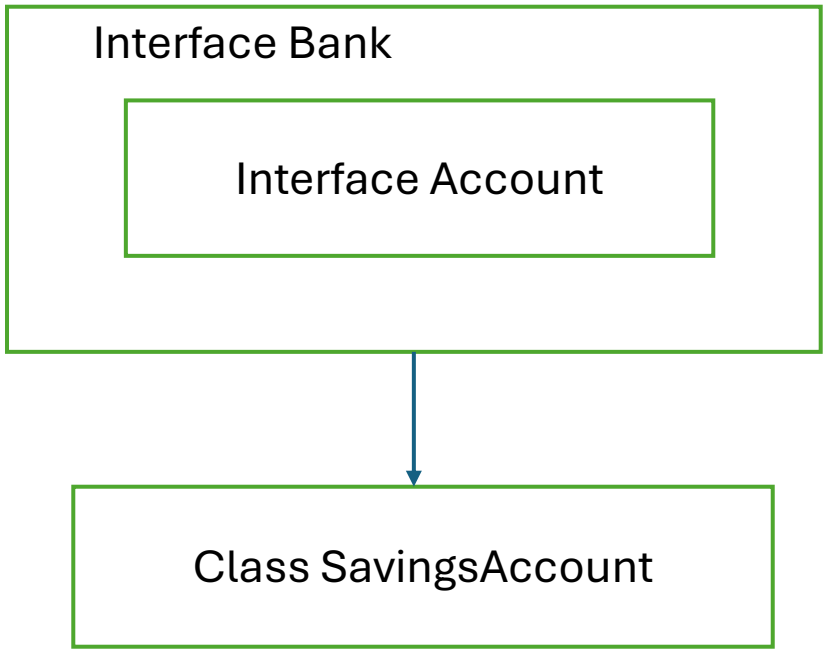
   interface Account {
      int MIN_BAL = 1000;       // attribute
      void details();
   }
}

class SavingsAccount implements Bank.Account {
   public void details() {
      System.out.println("Bank: " + Bank.BANK_NAME);
      System.out.println("Minimum Balance: ₹" +
MIN_BAL);
   }
}
```

```
public class Mainclass{
   public static void main(String[] args) {
      Bank.Account acc = new SavingsAccount();
      acc.details();
   }
}
```

Output:

Bank: City Bank
Minimum Balance: ₹1000



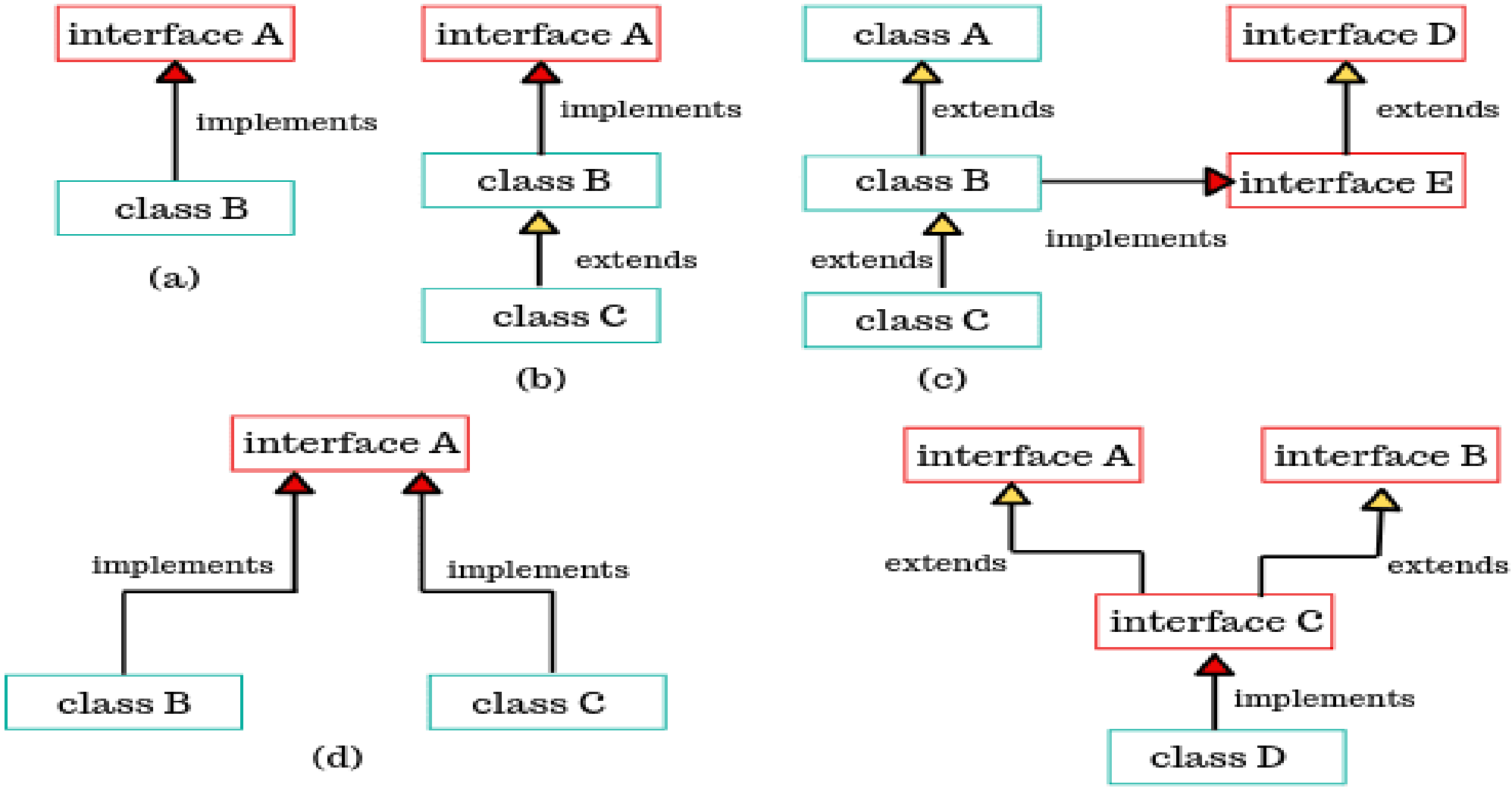


Fig: Various forms of interface implementation

(a) Interface implemented by a single class

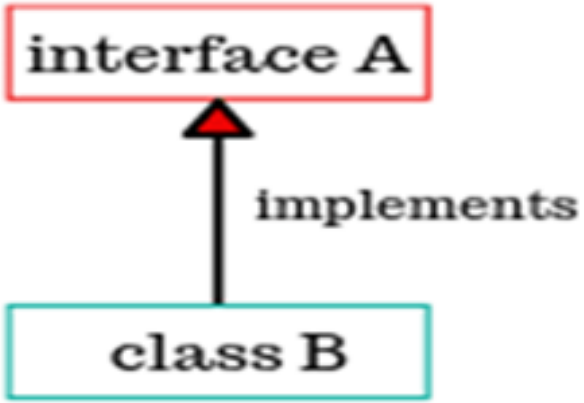
```
interface A {
   void display();
}

class B implements A {
   public void display() {
      System.out.println("Class B implements interface A");
   }
}

public class ExampleA {
   public static void main(String[] args) {
      B obj = new B();
      obj.display();
   }
}
```

Output:

Class B implements interface A



(a)

(b) Class extends another class and implements an interface

```
// Interface A
interface A {
   void show();    // abstract method
}

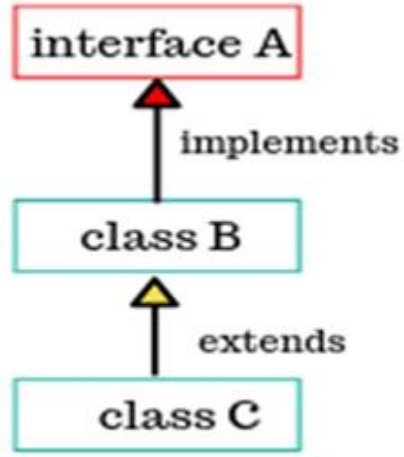
// Superclass C
class C {
   void message() {
      System.out.println("Class C method");
   }
}

// Subclass B extends C and implements interface A
class B extends C implements A {
   public void show() {        // definition of abstract method
      System.out.println("Class B implements interface A and
extends Class C");
   }
}
```

```
public class ExampleB {
   public static void main(String[] args) {
      B obj = new B();              // object of subclass
      obj.message();                // inherited from C
      obj.show();                   // implemented from A
   }
}
```

Output:

```
Class C method
Class B implements interface A and exte
```



(b)

```java
// Interface E with one abstract method
interface E {
   void display();
}

// Interface D extends E (inherits display() from E)
interface D extends E {
}

// Class C implements D (so it must define display())
class C implements D {
   public void display() {
      System.out.println("Display method implemented in
class C");
   }
}

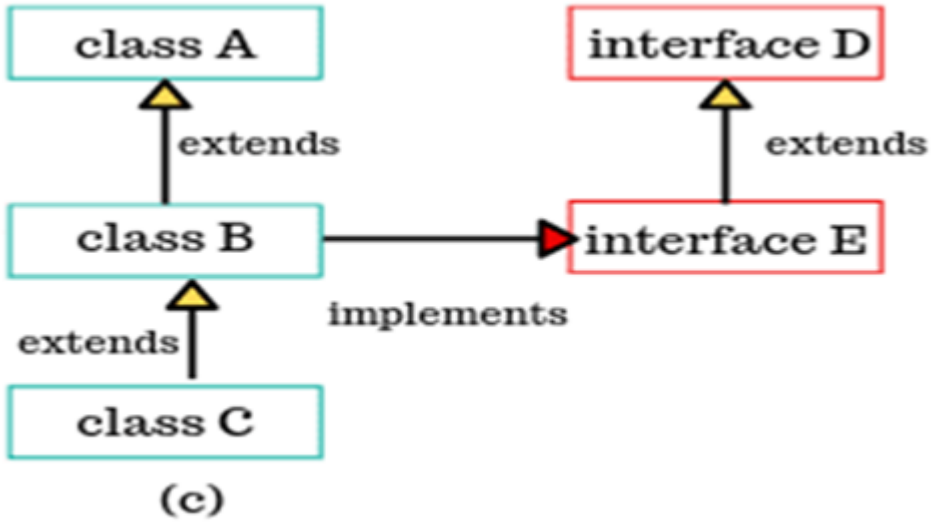
// Class B extends C (inherits display() from C)
class B extends C {
}

// Class A extends B (inherits display() from B)
class A extends B {
}

// Main class
public class ExampleC {
   public static void main(String[] args) {
      A obj = new A();   // A inherits everything from B → C → D → E
      obj.display();     // calling display() method from C
   }
}
```

Output:

Display method implemented in class C



(c)

(d) **Multiple classes implementing the same interface**

```
// Interface A
interface A {
   void display();
}

// Class B implements A
class B implements A {
   public void display() {
      System.out.println("Class B implements interface A");
   }
}

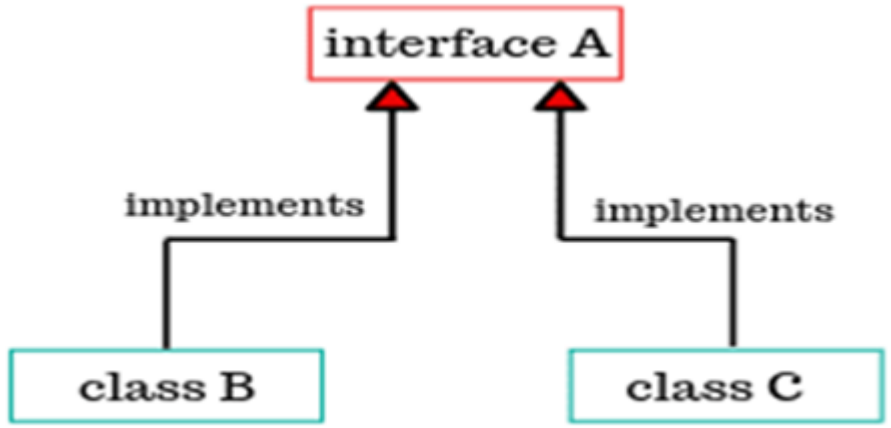
// Class C implements A
class C implements A {
   public void display() {
      System.out.println("Class C implements interface A");
   }
}
```

```
// Main class
public class ExampleD {
   public static void main(String[] args) {
      A obj1 = new B();   // interface reference to class B object
      obj1.display();

      A obj2 = new C();   // interface reference to class C object
      obj2.display();
   }
}
```

Output:

Class B implements interface A
Class C implements interface A



(d)

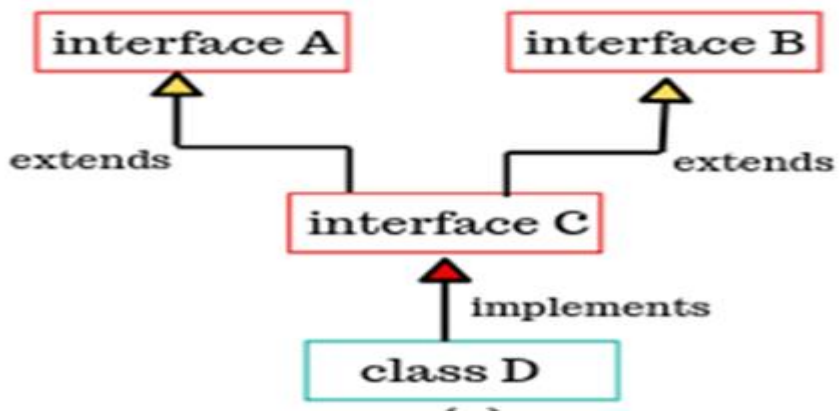
(e) Multiple interfaces extended and implemented by a class

```
// Interface A
interface A {
   void methodA();
}

// Interface B
interface B {
   void methodB();
}

// Interface C extends both A and B
interface C extends A, B {
   void methodC();
}
// Class D implements C (must define all methods from A, B, and C)
class D implements C {
   public void methodA() {
      System.out.println("Method from Interface A");
   }
 }
public void methodB() {
      System.out.println("Method from Interface B");
public void methodC() {
      System.out.println("Method from Interface C
implemented in Class D");
   }
}

// Main class
public class ExampleE {
   public static void main(String[] args) {
      D obj = new D();     // create object of class D
      obj.methodA();     // method from interface A
      obj.methodB();     // method from interface B
      obj.methodC();     // method from interface C
   }
}
```



Package

A **package** is a way to **group related classes, interfaces, and sub-packages** together — like folders in your computer.

Example in Real Life:

Imagine your computer has:

```
Documents/
└── School/
    └── Notes.txt
```

Similarly, in Java:

```
mypackage/
└── Example.java
```

Here, mypackage is like the **folder name (package)**, and Example.java is the **file (class)** inside it.

Advantages of Java Packages





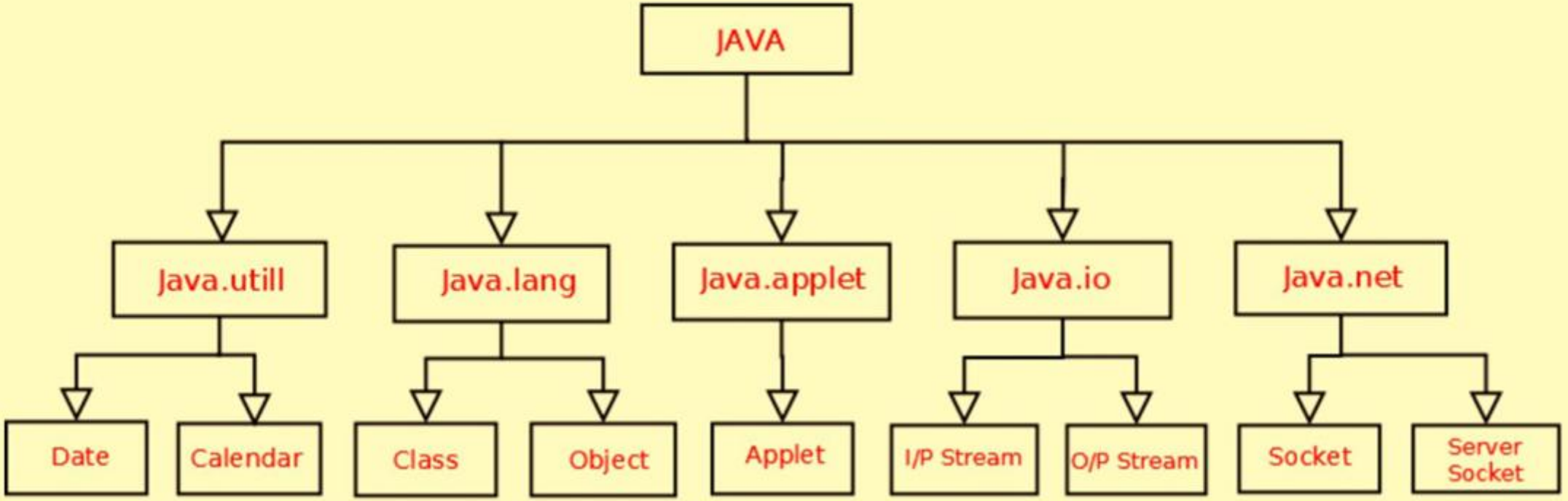
There are two types of packages in java:

1. **Built-in packages**

2. **User defined packages**

- Types of package:
  - 1) User defined package: The package we create is called user-defined package.
  - 2) Built-in package: The already defined package like java.io.*, java.lang.* etc are known as built-in packages.

1.Built-in Packages in Java



2. User Defined Packages :

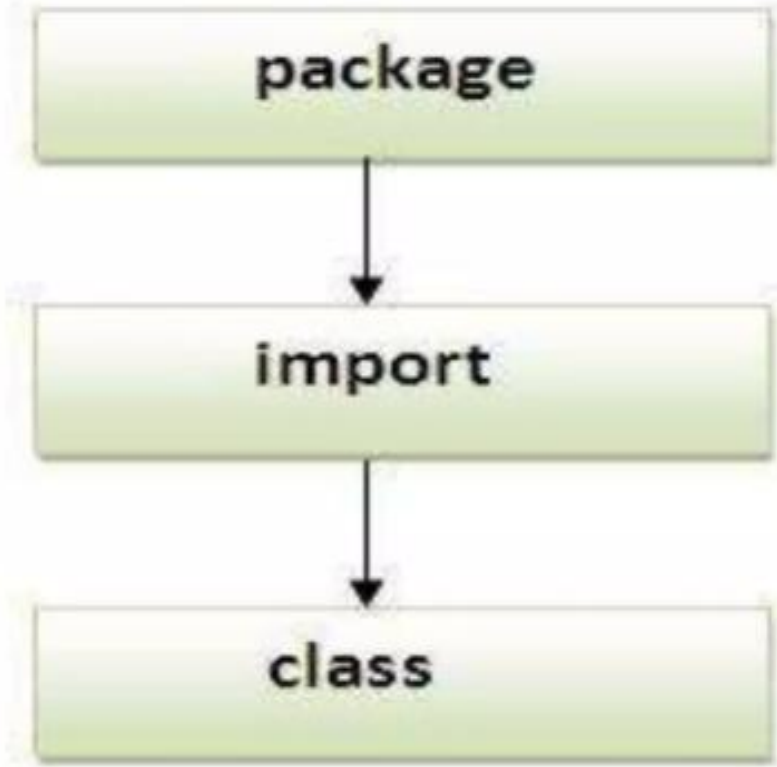
The users of the Java language can also create their own packages. They are called user-defined packages. User defined packages can also be imported into other classes & used exactly in the same way as the Built in packages.

i) Creating User Defined Packages

Syntax :

```
package packageName;
public class className
{
        - - - - - - - - - - - - - -
        // Body of className
        - - - - - - - - - - - - - -
}
```

Sequence of the program must be package then import then class.



Steps to create and user defined package in Java

1.Create a folder in desktop.

1. **Inside it create another folder for package.** (e.g., **package packagename;eg: package mypackage;)** packagename and foldername must be **same**.

2.**Create the file** inside a **folder** and **compile** it. **javac** filename.java

3. **Use the package** in another file by importing it with the import statement (e.g. **import packagename.***;).

5. **Compile and Run the main program** normally, and the Java compiler will locate the class inside the package.

Creating a Package

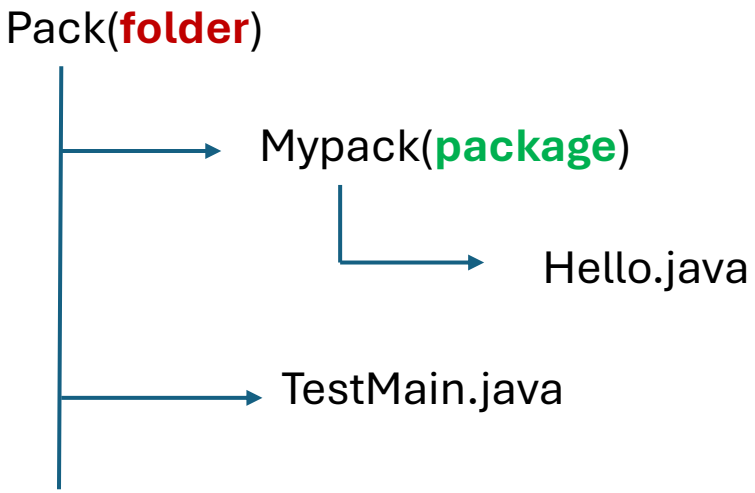
Syntax

package **packagename;**

Example

```
// File: mypack/Hello.java
package mypack;

public class Hello {
    public void show() {
        System.out.println("Hello from mypack package!");
    }
}
```



1.Create a folder named **pack** in desktop
2.Create a subfolder named **Mypack**.
Inside it save a file **Hello.java** and compile it.
3.In main folder save **TestMain.java** program.Now compile and run the program

Main File:

```
// File: TestMain.java
import mypack.*;

public class TestMain {
    public static void main(String[] args) {
        Hello h = new Hello();
        h.show();
    }
}
```

Compile & Run:

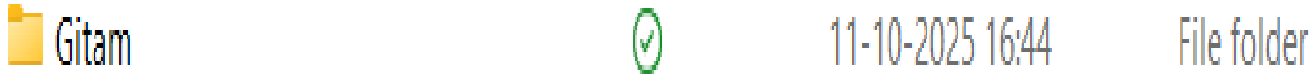
```
javac mypack/Hello.java
javac TestMain.java
java TestMain
```

Example1:Create a package?

Gitam

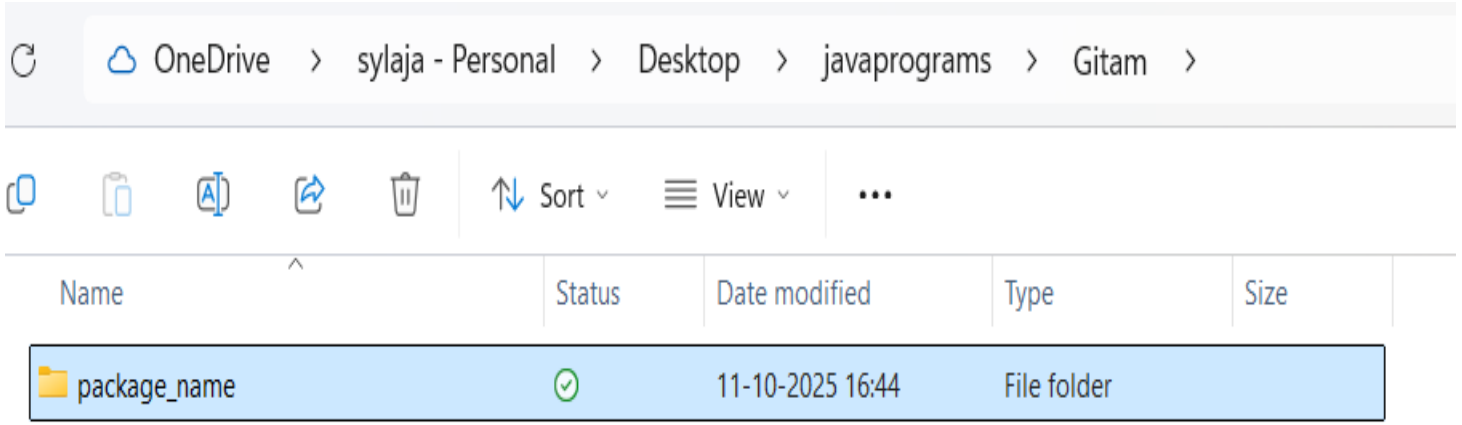
```
├── package_name
│     ├── ClassOne.java
│     └── ClassOne.class
├── package_one
│     ├── ClassTwo.java
│     └── ClassTwo.class
├── Testing.java
└── Testing.class
```

1.Create main folder Gitam

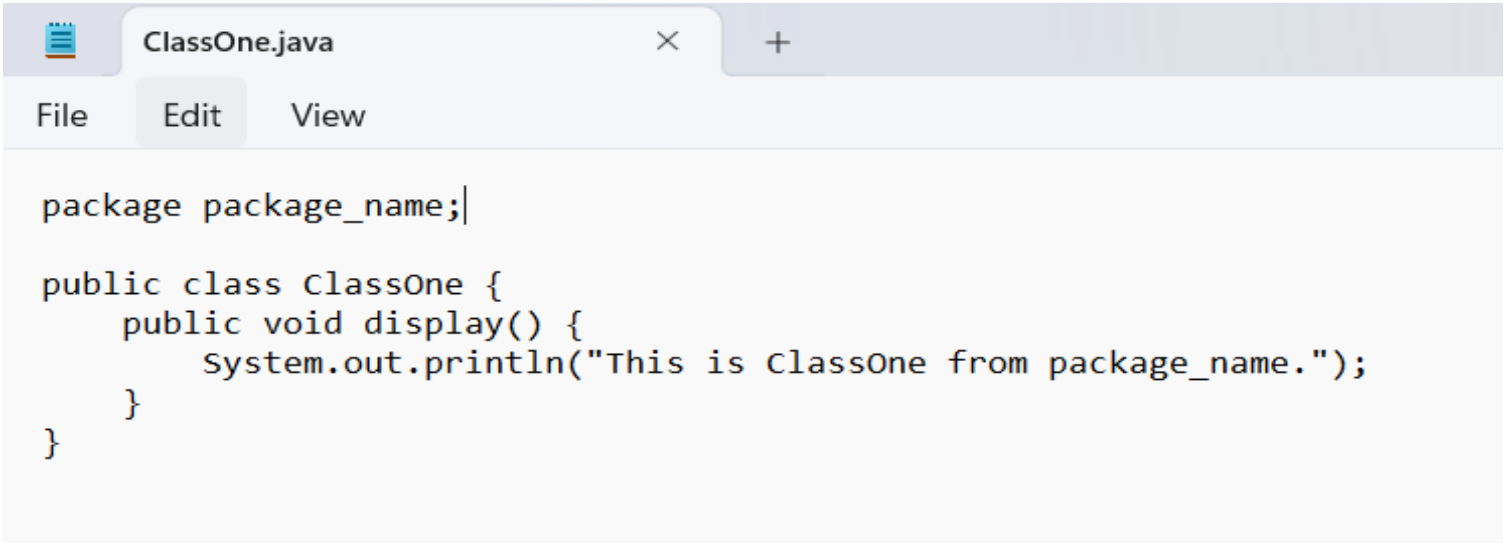


2.Create a subfolder(package_name) inside Gitam .

Note: package name and folder name must be same

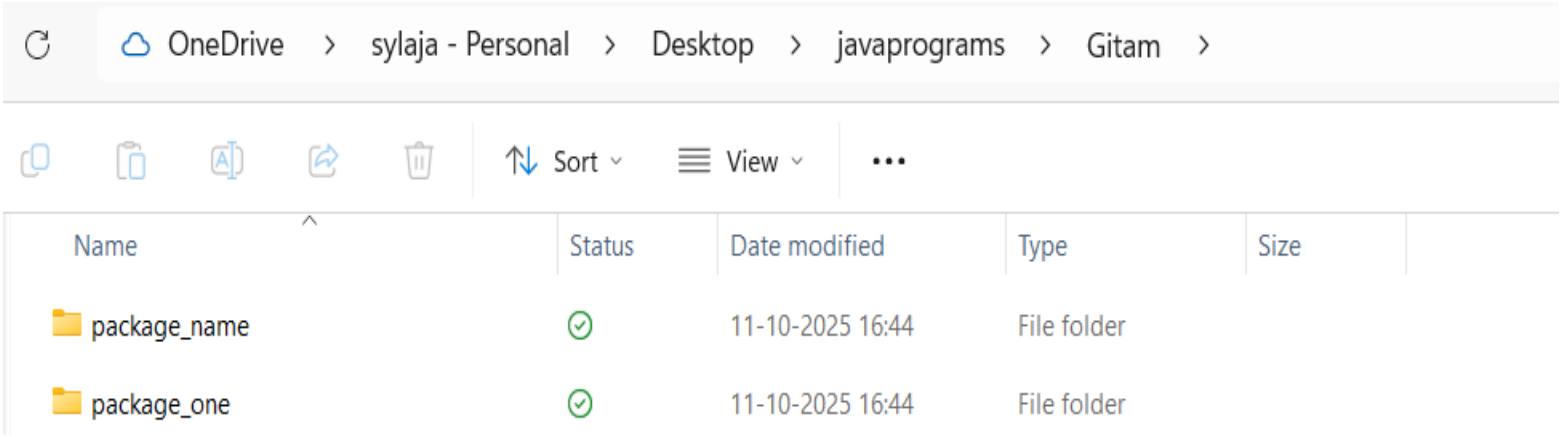


Now type a `ClassOne.java program in notepad and save in subfolder package_name`



Compile: javac ClassOne.java

3.Create another subfolder package_one inside Gitam



inside type a program in notepad and save it as ClassTwo.java

ClassTwo.java

File Edit View

```
package package_one;

public class ClassTwo {
    public void show() {
        System.out.println("This is ClassTwo from package_one.");
    }
}
```

Compile:javac ClassTwo.java

4.Inside main folder Gitam create a class main file(in notepad) Testing.java

Testing.java

File Edit View

```
import package_name.ClassOne;
import package_one.ClassTwo;

public class Testing {
    public static void main(String[] args) {
        ClassOne obj1 = new ClassOne();
        ClassTwo obj2 = new ClassTwo();

        obj1.display();
        obj2.show();
    }
}
```

Note: also give as

import **package_name.*;**

import **package_one.*;**

5.Compile and run the program in main folder(Gitam)

```
C:\Users\sylaj\OneDrive\Desktop\javaprograms\Gitam>javac Testing.java

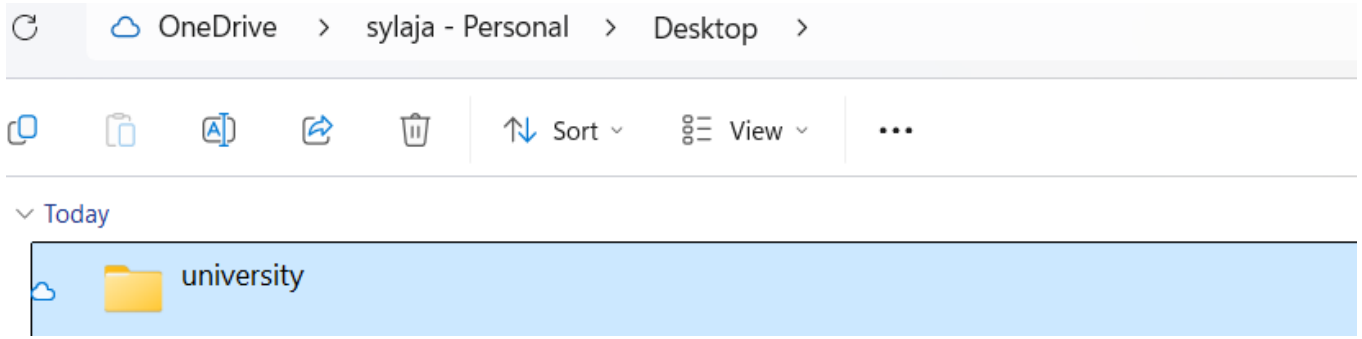
C:\Users\sylaj\OneDrive\Desktop\javaprograms\Gitam>java Testing
This is ClassOne from package_name.
This is ClassTwo from package_one.
```

Example2:Create a package

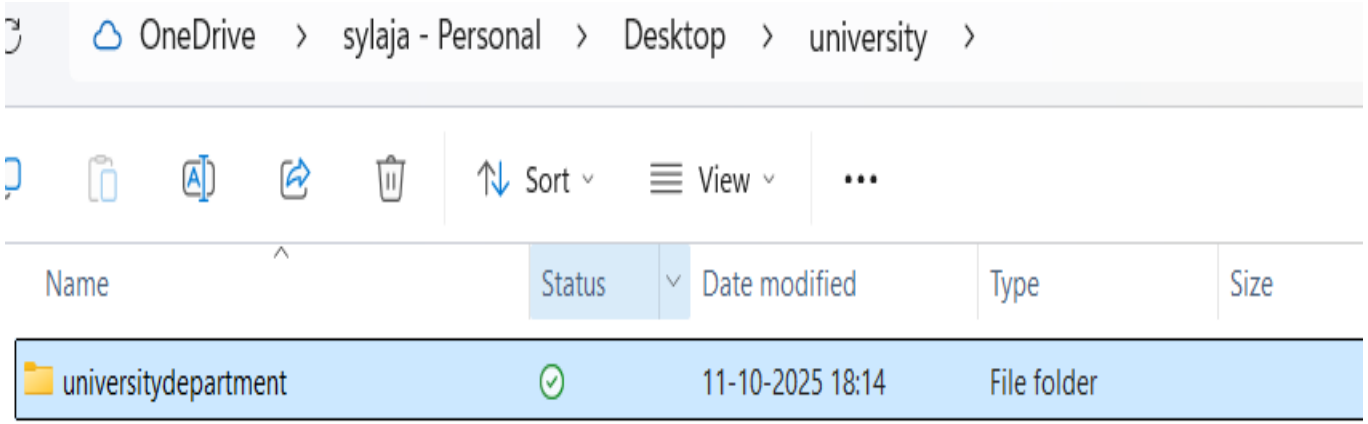
📁 Folder structure

```
university/
 ├── department/
 │      └── Department.java
 ├── student/
 │      └── Student.java
 └── MainApp.java
```

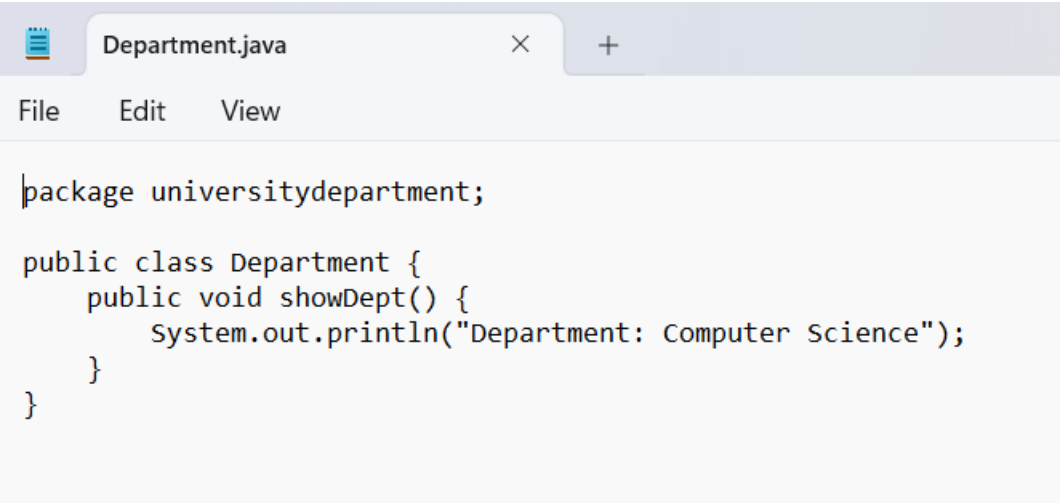
1.Create a newfolder in desktop named University



2.Inside university create new sub folder universitydepartment

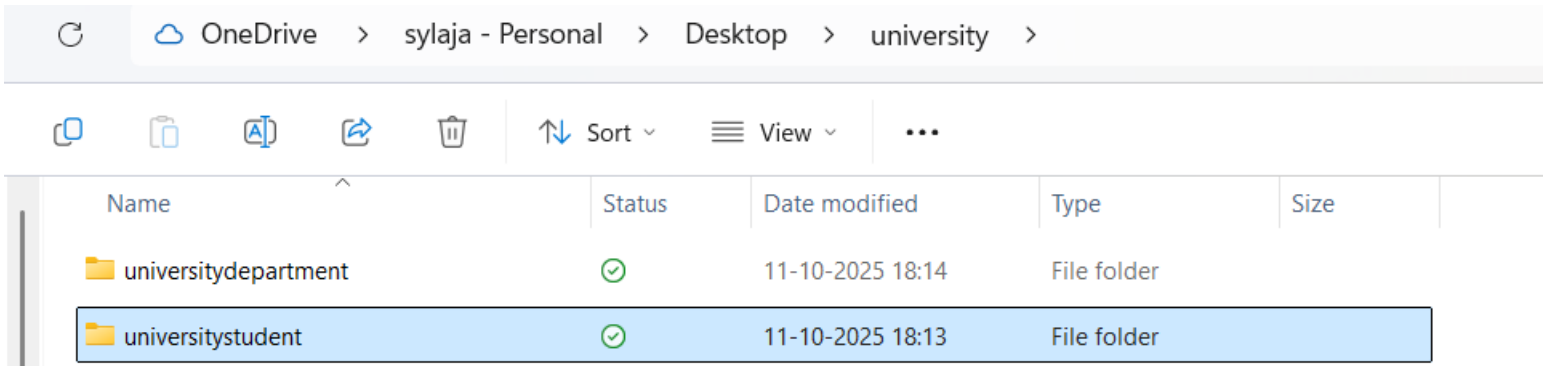


Open a notepad and create class file Department.java inside university.department folder

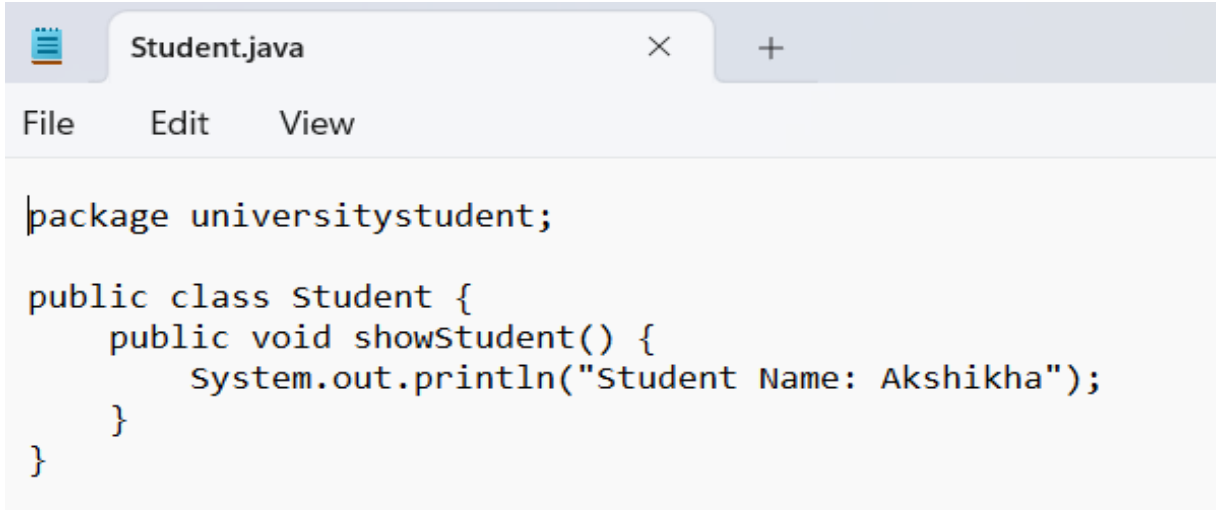


Compile it

3.Inside university folder create another folder universitystudent



Open notepad and create a class file Student.java in universitystudent folder



Compile it.

4.Go to main folder university and create a main class MainApp .java

MainApp.java

File Edit View

```java
import universitydepartment.Department;
import universitystudent.Student;

public class MainApp {
    public static void main(String[] args) {
        Department d = new Department();
        Student s = new Student();

        System.out.println("=== University Details ===");
        d.showDept();
        s.showStudent();
    }
}
```

5.Goto command prompt in main folder university ,compile and run the classfile Mainapp

```
C:\Users\sylaj\OneDrive\Desktop\university>javac MainApp.java

C:\Users\sylaj\OneDrive\Desktop\university>java MainApp
=== University Details ===
Department: Computer Science
Student Name: Akshikha

C:\Users\sylaj\OneDrive\Desktop\university>
```

Examples :Create user defined package

```
bank
├── accounts
│   ├── SavingsAccount.java
│   └── SavingsAccount.class
│
├── loans
│   ├── LoanAccount.java
│   └── LoanAccount.class
│
├── BankMain.java
└── BankMain.class
```

```
ecommerce
├── product
│   └── Product.java
├── customer
│   └── Customer.java
└── MainStore.java
```

```
vehicle
├── car
│   └── Car.java
├── bike
│   └── Bike.java
└── VehicleMain.java
```

```
university
├── department
│   ├── Department.java
│   └── Department.class
├── student
│   ├── Student.java
│   └── Student.class
├── MainApp.java
└── MainApp.class
```

```
hospital
├── doctor
│   └── Doctor.java
├── patient
│   └── Patient.java
└── HospitalMain.java
```

```
library
├── books
│   └── Book.java
├── members
│   └── Member.java
└── LibraryMain.java
```

Static import in package

- **static import** is a feature in Java that allows you to **access static members (methods or variables)** of a class **directly without using the class name**.
- It helps make the code cleaner and more readable by **removing repetitive class name references**.

Syntax to import **all static members**:

```
import static package_name.class_name.*;
```

This can be achieved both in **pre-defined package** and **user-defined package.**

Static import in Pre-defined package

is achieved using

1.Using **Math class**

2.Using **static variables**

Note:

- Introduced in **Java 5**.
- Makes code concise but **should be used carefully** to avoid confusion (especially if multiple classes have static members with the same name).
- Can be used for constants (e.g., PI) or static methods (sqrt(), pow(), etc.).

Example 1: Using Math class

Program

```
import static java.lang.Math.*;

class TestStaticImport {
   public static void main(String[] args) {
      System.out.println(sqrt(25));          // No need to write Math.
      System.out.println(pow(2, 3));
   }
}
```

Output:

```
5.0
8.0
```

Example 2: Using static variables

```
import static java.lang.System.out;

class ExampleStaticImport {
    public static void main(String[] args) {
        out.println("Hello using static import!");
    }
}
```

Output:

Hello using static import!

`static import` in a user-defined package.

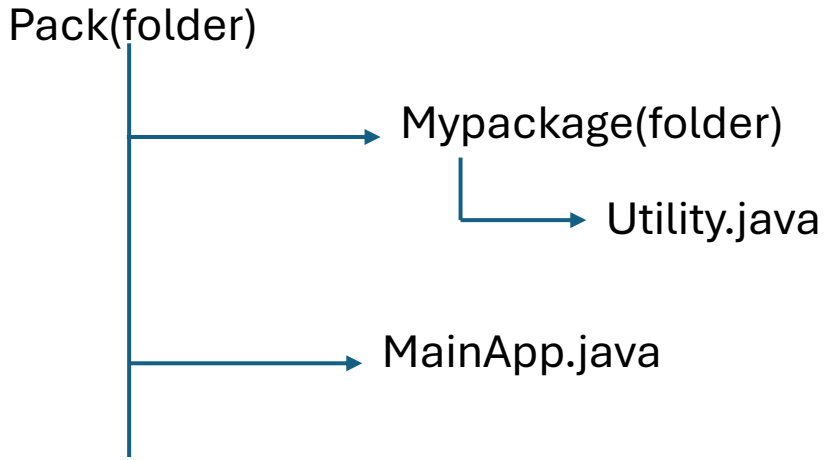
Step 1: Create a package(inside folder mypackage,save mypackage.java file)

```
package mypackage;

public class Utility {
    public static int square(int n) {
        return n * n;
    }

    public static void greet() {
        System.out.println("Welcome to Static Import Example!");
    }

    public static final double PI = 3.14159;
}
```



Step 2: Use static import in another class

Program

```
import static mypackage.Utility.*;   // static import of all static members

class MainApp {
    public static void main(String[] args) {
        greet();                    // Direct call — no class name needed
        System.out.println("Square of 5: " + square(5));
        System.out.println("Value of PI: " + PI);
    }
}
```

Compile and run Mainapp.java

Output:

```
Welcome to Static Import Example!
Square of 5: 25
Value of PI: 3.14159
```

FILE I/O (Input and Output in Java)

- **File I/O** means **reading data from a file** and **writing data to a file**.
- Java uses the `java.io` package

Stream

- A **stream** is a **sequence of data** that flows from one place to another.
- There are **two main types of streams**:

1. **InputStream** → Used to **read** data from a source (like a file).
2. **OutputStream** → Used to **write** data to a destination (like a file).

Byte Streams

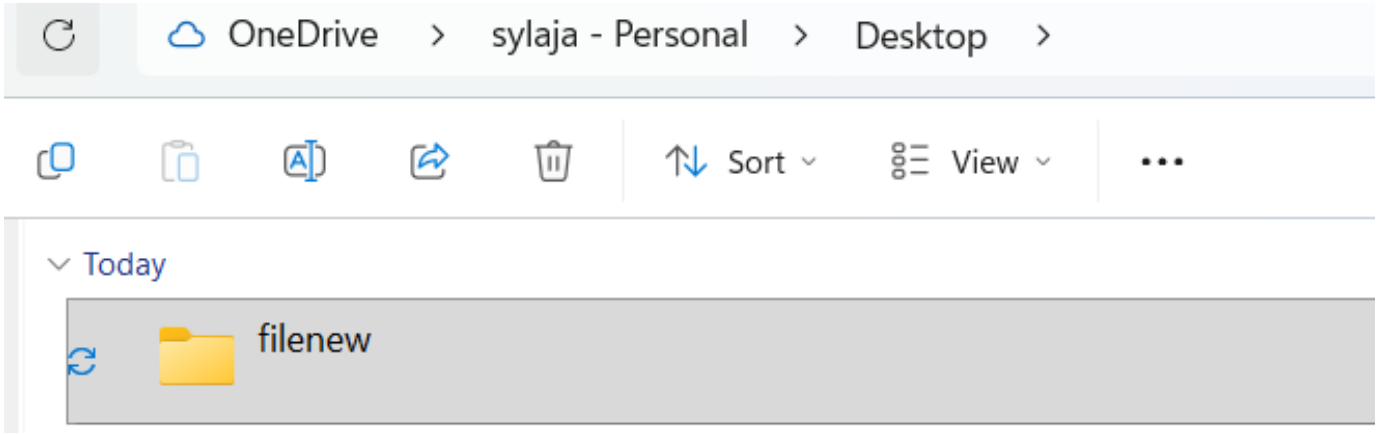
- Byte streams are used for **input and output of 8-bit bytes**.
- They are mainly used to handle **binary files** such as images, audio, and text files.
- The **two** commonly used **classes** are:
  - **`FileInputStream`** → for **reading** bytes from a file.
  - **`FileOutputStream`** → for **writing** bytes to a file.

Steps to create a file and to read data from a file

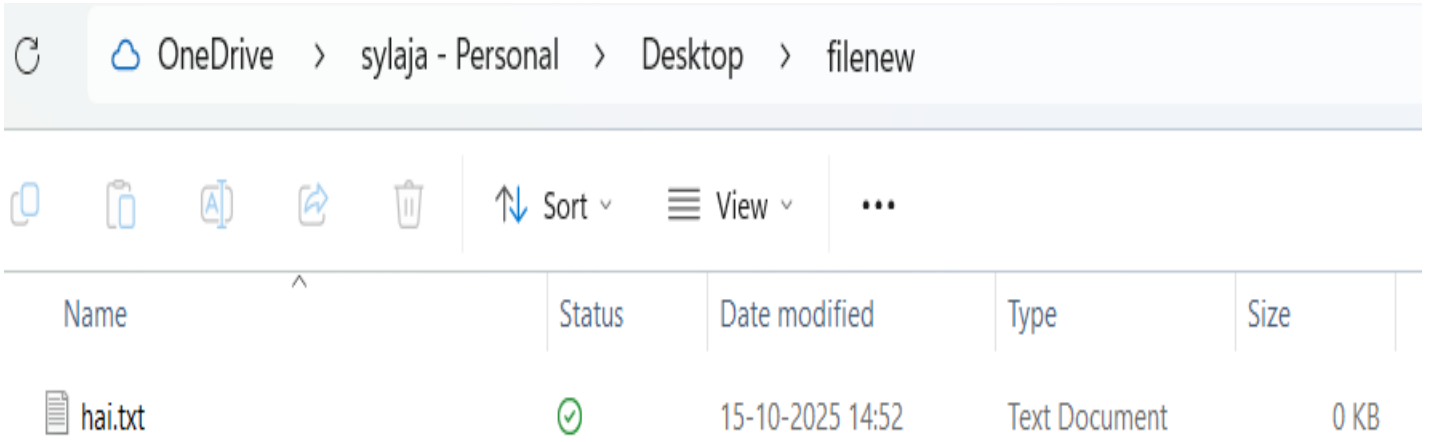
1. Create a **folder** named **filenew**
2. Create a **text document hai.txt** inside folder filenew and write contents.
3. Save **class file fio1.java** inside folder filenew
4. **Compile and run** fio1.java file

Program to display a single character

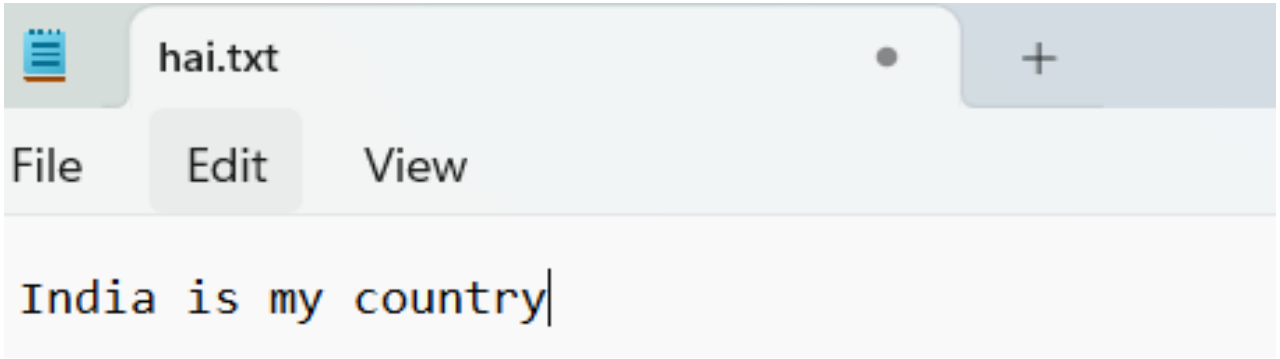
1.Create a folder filenew



2. Create a text document hai.txt inside filenew



3.Type the content



4.Create a classfile fio1.java

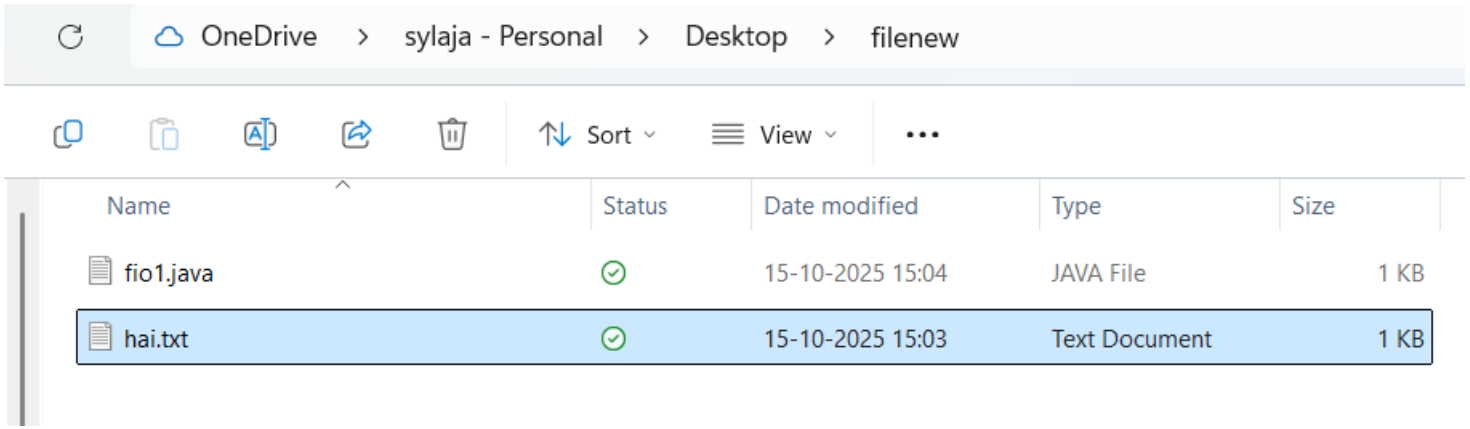
fio1.java

File Edit View

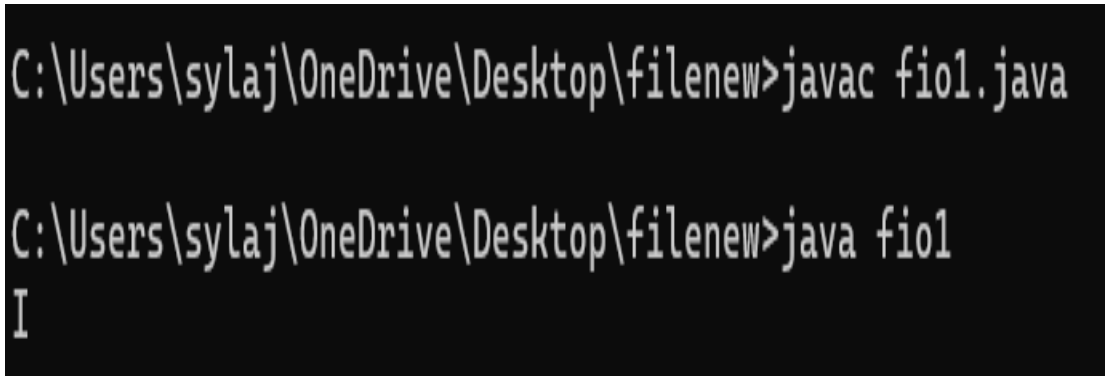
```
//read from the file
import java.io.*;
class fio1
   {
    public static void main(String ar[])throws Exception
        {

                FileInputStream in =new FileInputStream("hai.txt");
                char c;
                c=(char)in.read();
                System.out.println(c);
               }
}
```

5.Save inside filenew folder as fio1.java



6.Compile and run the program: It displays a single character



Example:Program to read a single character from a file(procedure shown in above diagram)

```
//read from the file
import java.io.*;

class fio1 {
   public static void main(String args[]) throws Exception {
      FileInputStream in = new FileInputStream("hai.txt");       // open file for reading
      char c;
      c = (char) in.read();                    // read one byte and cast it to a character
      System.out.println(c);    // print the character
      in.close();                    // close the file
   }
}
```

Output

If the file hai.txt contains:

Hello

The output will be:

H

Step-by-Step Execution

1. `import java.io.*;` → Imports all input/output classes.
2. `FileInputStream in = new FileInputStream("hai.txt");`
Opens the file `hai.txt` for reading.
3. `in.read()` → Reads one byte from the file.
4. `(char) in.read()` → Converts the byte into a character.
5. `System.out.println(c);` → Displays the character on the screen.
6. `in.close();` → Closes the file to free resources.

Output

If the file hai.txt contains:

Hello

The output will be:

H

Note:

read() returns **-1** when the end of the file (EOF) is reached.

To read the entire file, you should use a **loop** like this:

```
int i;
while((i = in.read()) != -1)
   System.out.print((char)i);
```

Example:Program to read entire content from the file(same procedure as above)

```
// Importing the java.io package to use classes for file input and output operations
import java.io.*;

// Class declaration
class fio2
{
    // Main method where program execution begins
    public static void main(String args[]) throws Exception
    {
        // Create a FileInputStream object to read data from the file "hai.txt"
        // The file must exist in the same folder where the program runs
        FileInputStream in = new FileInputStream("hai.txt");

        int c; // Variable to store each character read from the file

       // Loop runs until end of file (-1) is reached
        while ((c = in.read()) != -1)
        {
        // Convert integer value 'c' to a character and print it
            System.out.println((char)c);
        }
    }
}
```

Output

If the file hai.txt contains:

Hello

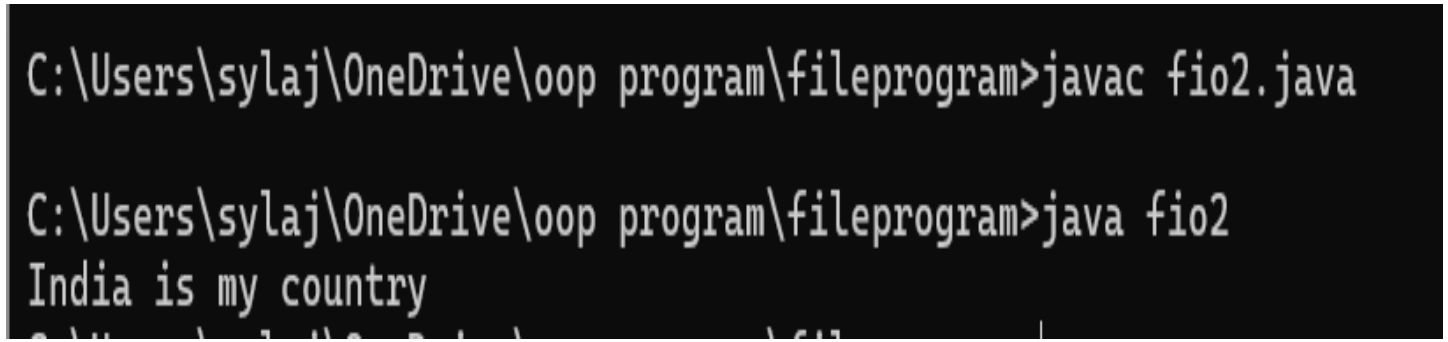
The output will be:

Hello

Output: Create hai.txt and enter the content



Compile and run the program.It displays entire content



Example:Program to write one character to file

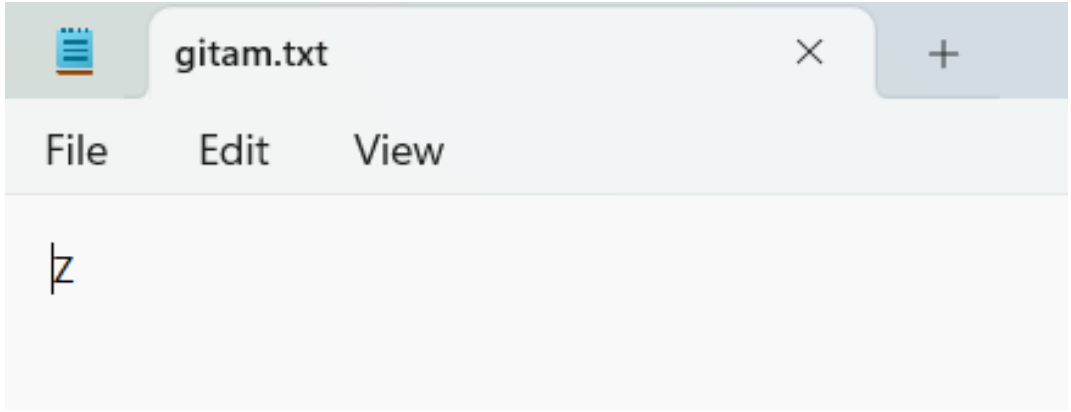
```
import java.io.*;
import java.util.*;
class fio3
  {
   public static void main(String args[])throws Exception
          {
                  // Create a FileOutputStream object to write data into a file named "gitam.txt"
                   // If the file doesn't exist, it will be created automatically
                   FileOutputStream out =new FileOutputStream("gitam.txt");
                   Scanner scr=new Scanner(System.in);
                   int c;
                  // Read first character of the entered input
                   // next() reads a string, and charAt(0) takes the first character
                  /*c=scr.nextInt();
                   char c; // (or)*/
                   c=scr.next().charAt(0);
                  // Write the character into the file as its byte value
                   out.write(c);
          }
}
```

Output:

```
C:\Users\sylaj\OneDrive\oop program\fileprogram>javac fio3.java

C:\Users\sylaj\OneDrive\oop program\fileprogram>java fio3
Zero
```

Go to folder and now you can see **gitam.txt** created automatically.Open it you see a **single character**



Example: program **reads all characters from the file "hai.txt" and writes that content into another file named "output.txt".**

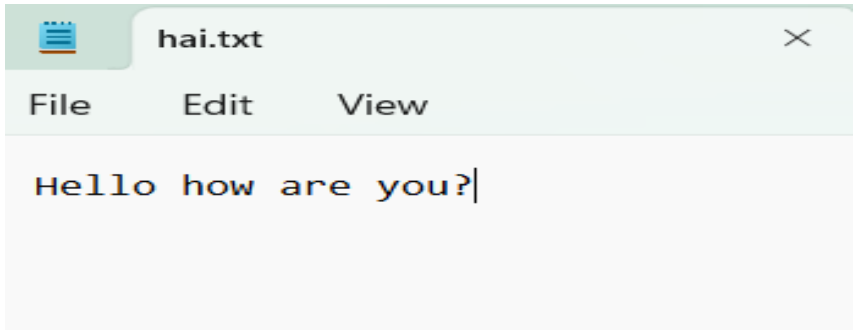
Program

```
import java.io.*;
class fio5
{
   public static void main(String args[]) throws Exception
   {
      // Open "hai.txt" for reading
      FileInputStream in = new FileInputStream("hai.txt");
      // Create or open "output.txt" for writing
      FileOutputStream out = new FileOutputStream("output.txt");

      int c; // Variable to hold each character read from the input file

      // Read each character from hai.txt until the end (-1 means end of file)
      while ((c = in.read()) != -1)
      {
         // Write each character read from hai.txt into output.txt
         out.write(c);
      }
  }
}
```

Output: create hai.txt and write content.

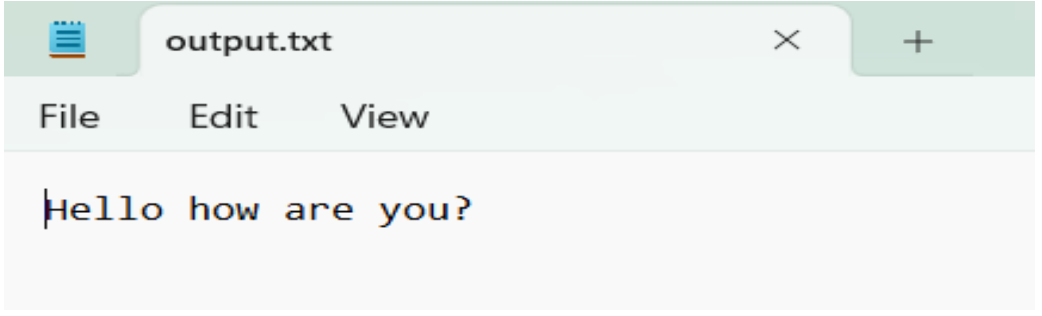


compile and run fio5.

```
C:\Users\sylaj\OneDrive\oop program\fileprogram>javac fio5.java

C:\Users\sylaj\OneDrive\oop program\fileprogram>java fio5
```

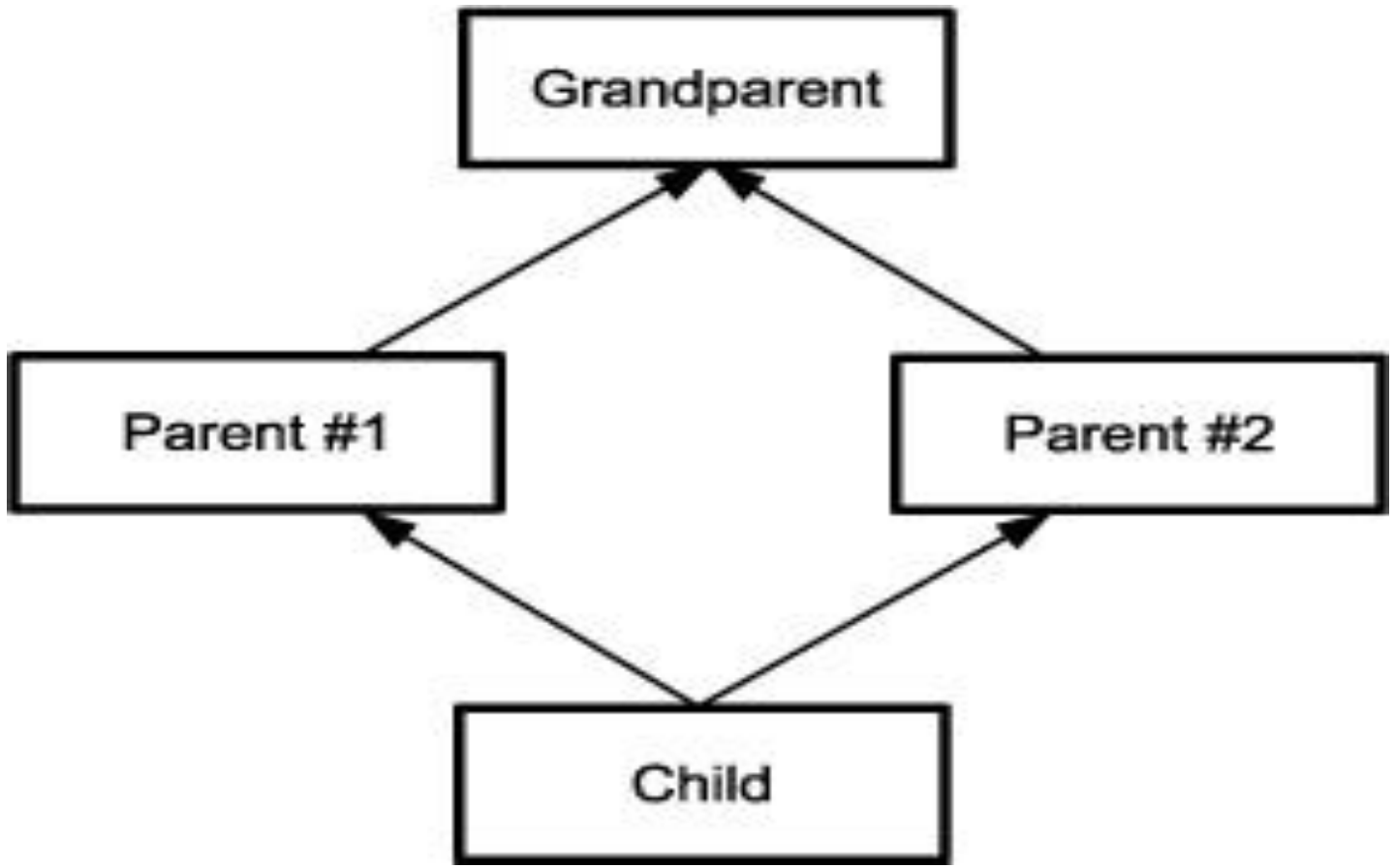
Automatically an output.txt is created in folder.When we open it same content in hai.txt will be in output.txt

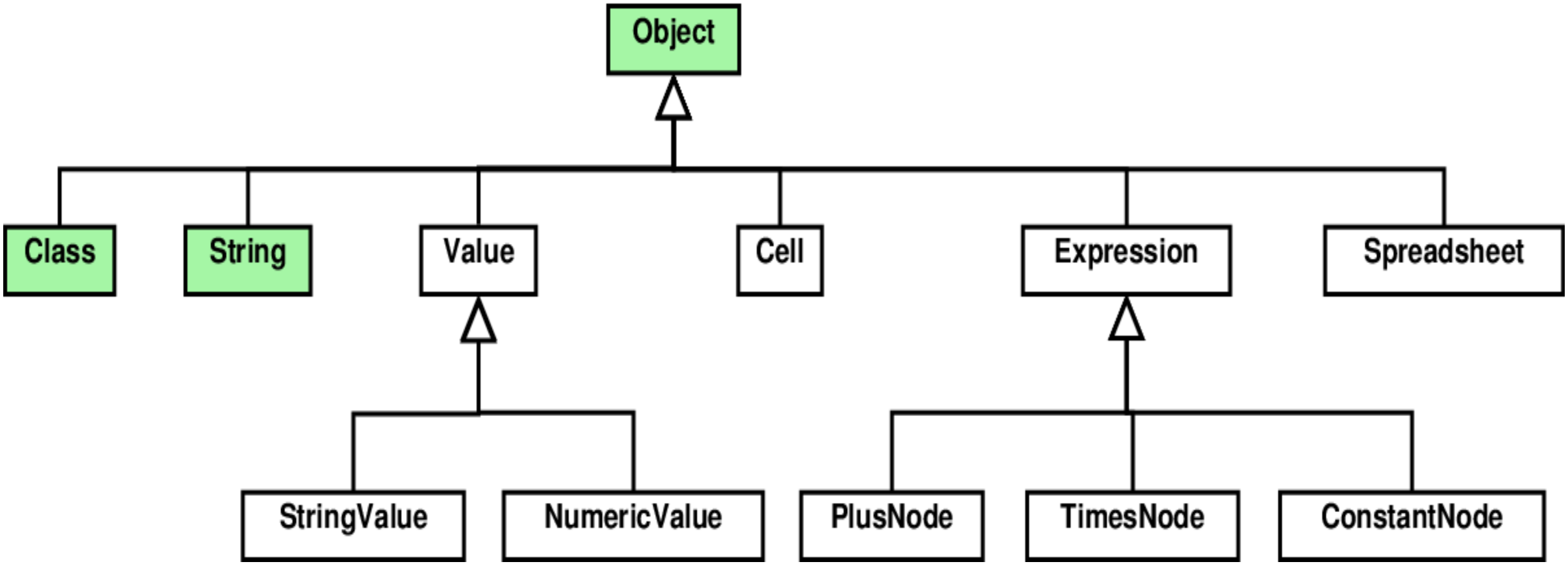


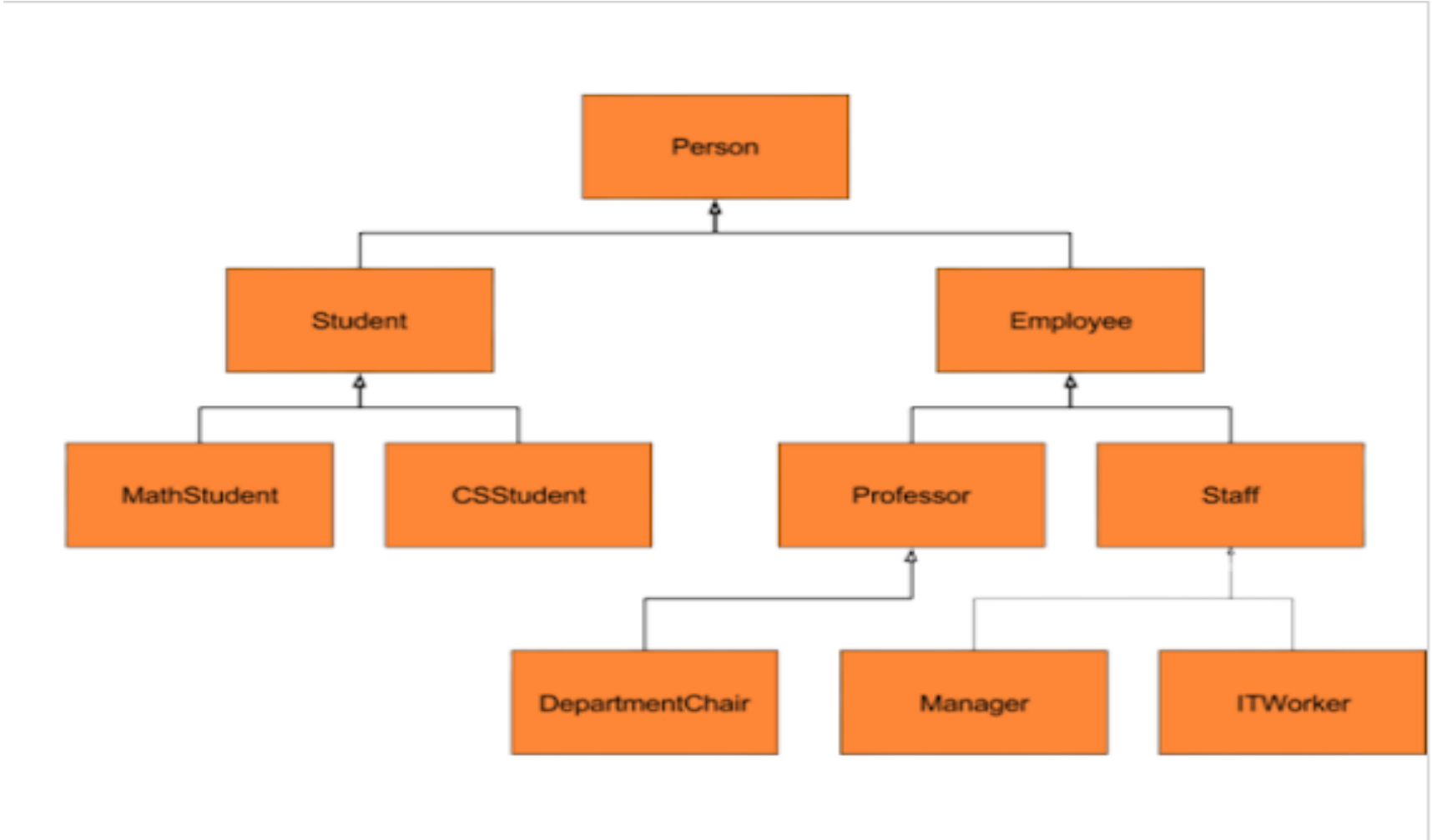
Applications of File I/O (Input/Output) in Java

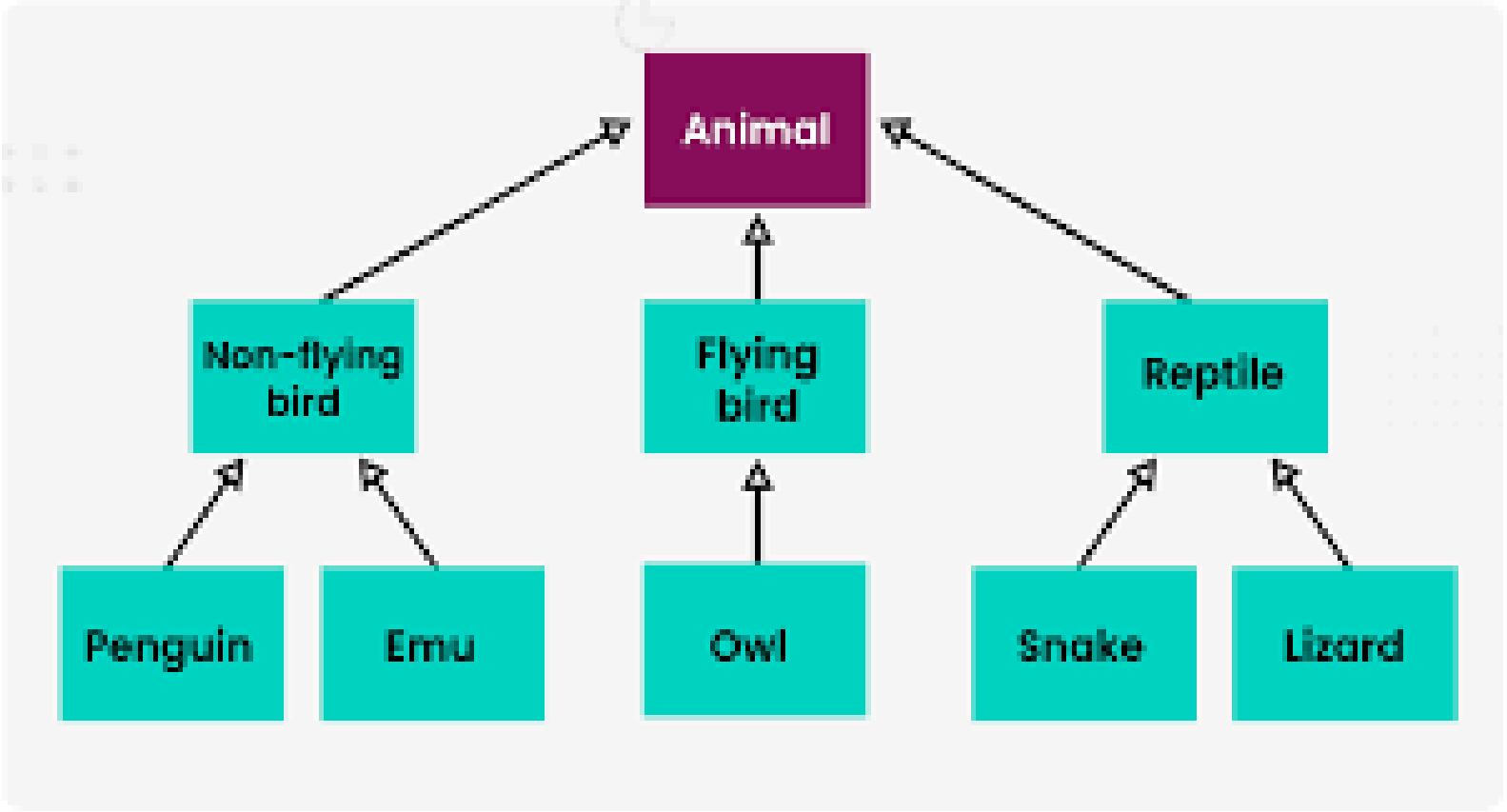
1.Data Storage and Retrieval:
Used to **store user data** (like login details, scores, or records) and **retrieve** it when needed — e.g., saving student records or user preferences.
2.Configuration Files:
Applications read settings or preferences (like theme, database path, or language) from configuration files using File I/O.
3.Logging and Debugging:
Used to **write logs** of system or application activities (errors, user actions) for debugging or monitoring.
4.Report Generation:
File I/O helps generate **text or CSV reports** for summaries, invoices, or results.
5.File Conversion:
Programs can read data from one file type and **write it into another** — e.g., converting .txt to .csv or .xml.
6.Backup and Restore: Used for **copying data** from one file to another for backup or recovery purposes.
7.Communication Between Programs:
Some applications **exchange data via files**, especially when direct communication is not possible (batch processing).

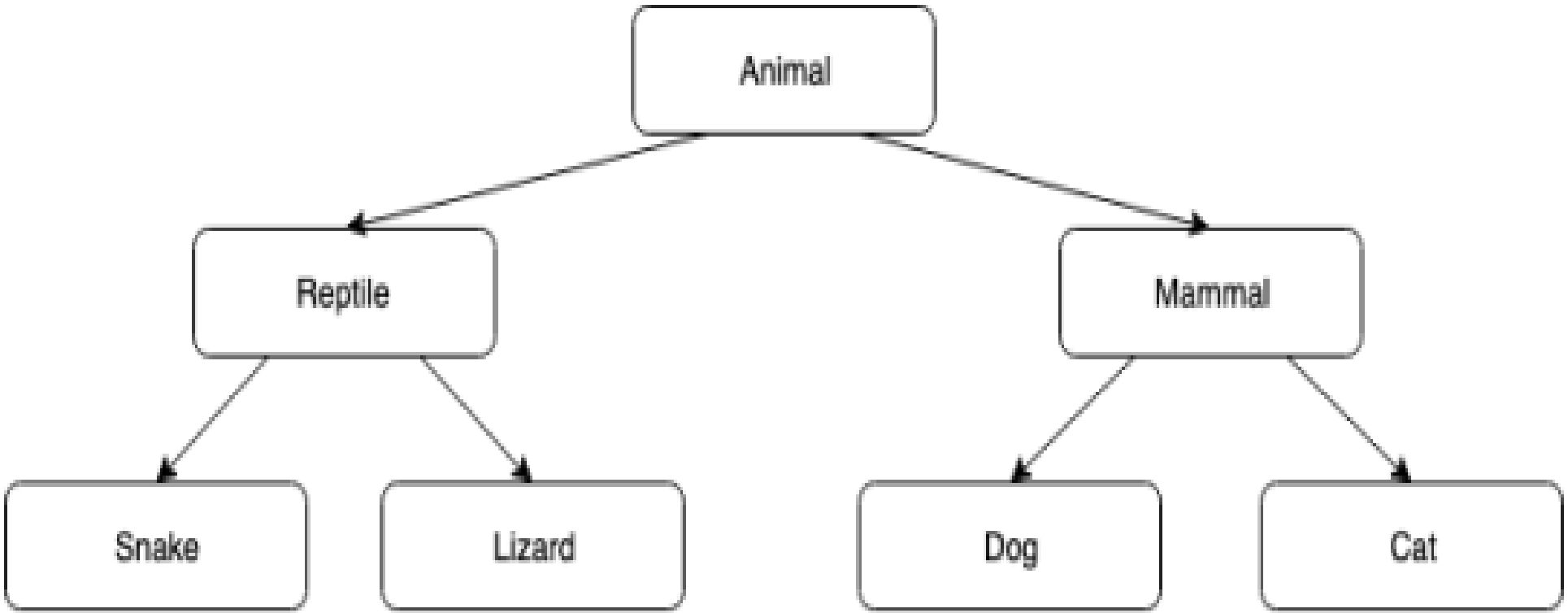
TYPES OF INHERITANCE


Grandparent
Parent #1
Parent #2
Child

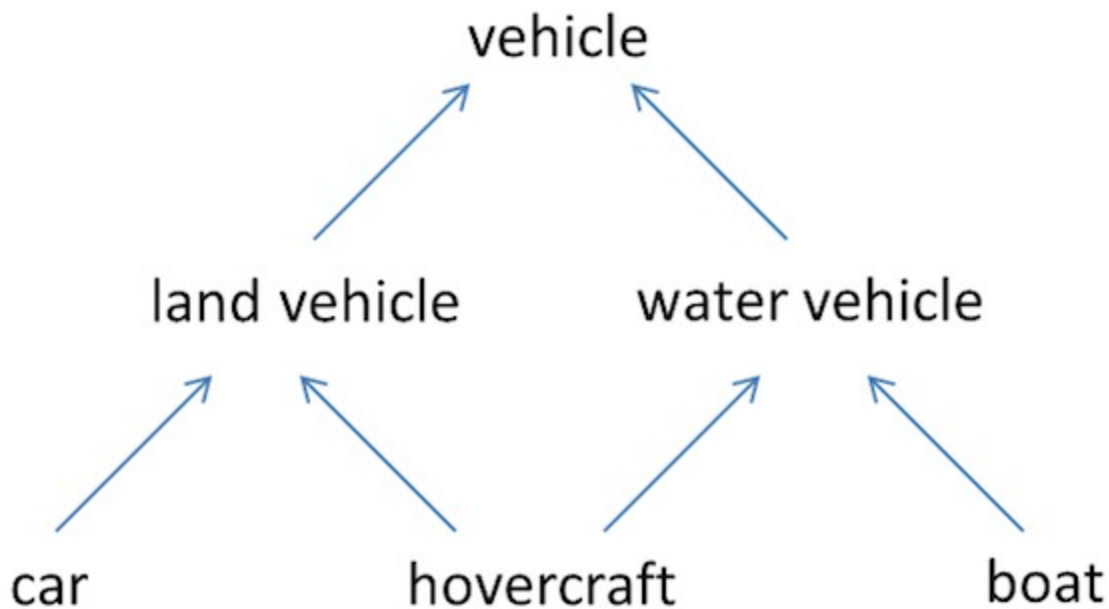

Object
Class
String
Value
Cell
Expression
Spreadsheet
StringValue
NumericValue
PlusNode
TimesNode
ConstantNode


Person
Student
Employee
MathStudent
CSStudent
Professor
Staff
DepartmentChair
Manager
ITWorker

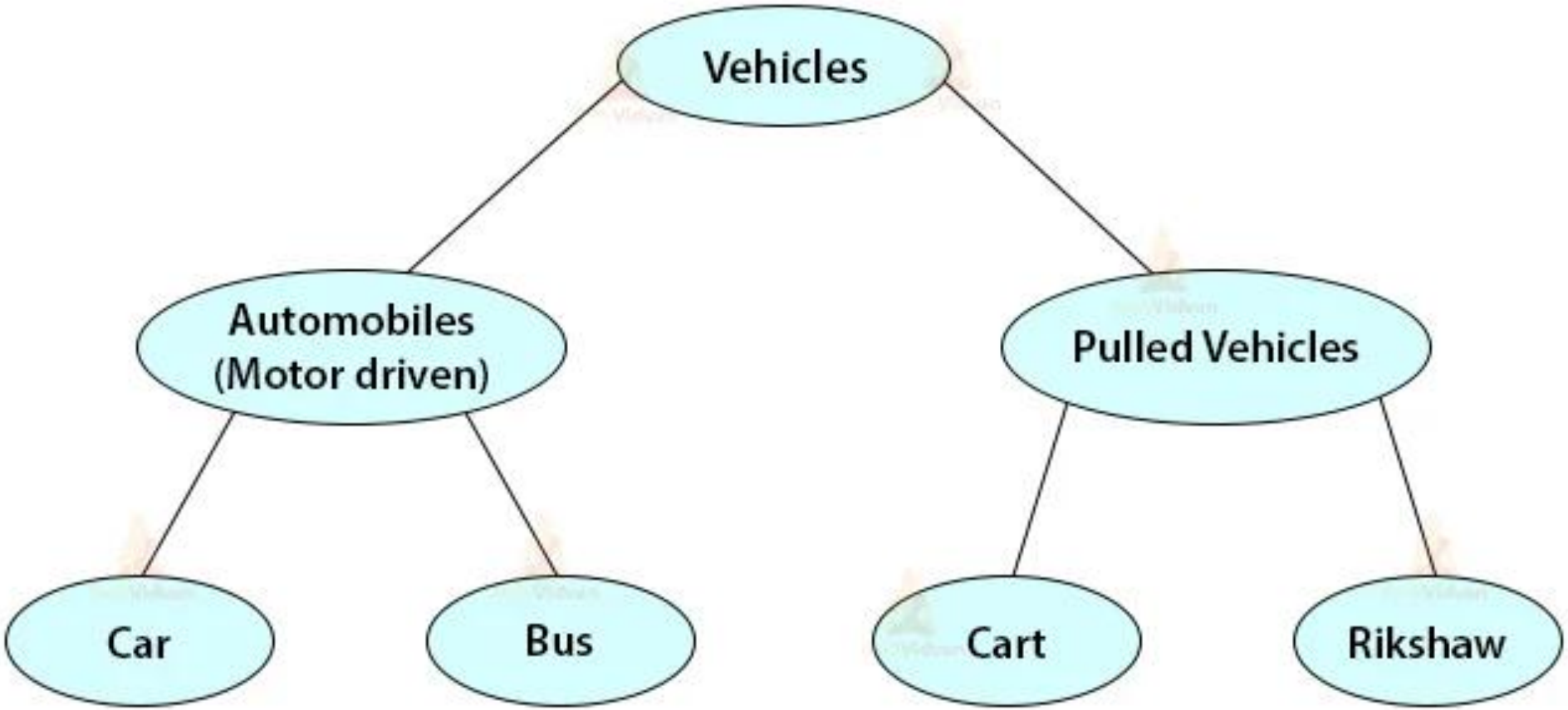

Animal
Non-flying bird
Flying bird
Reptile
Penguin
Emu
Owl
Snake
Lizard

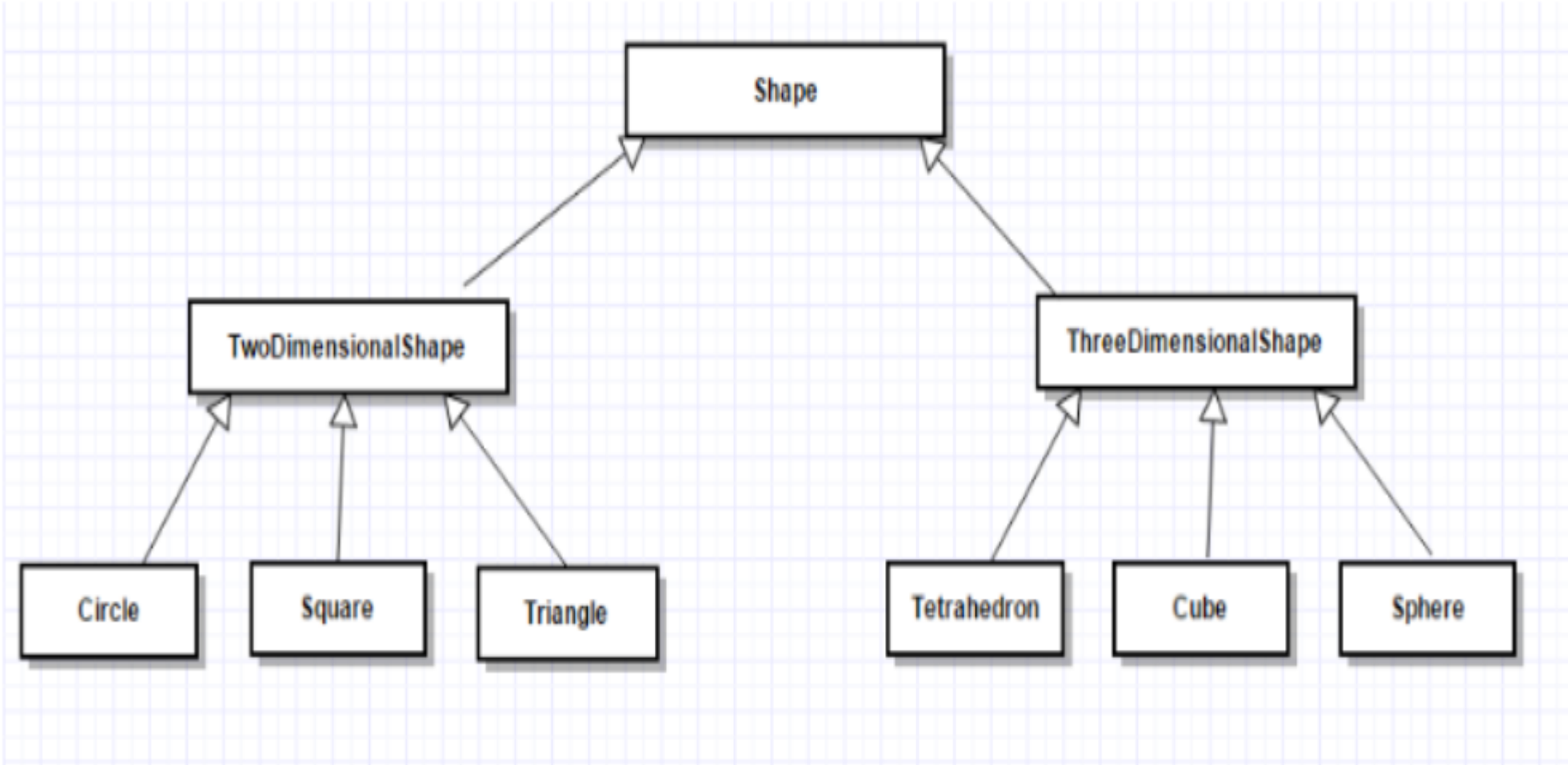


Typical introductory inheritance example. Does a good job introducing the idea of an *is-a* relationship, but not much else.


vehicle
land vehicle
water vehicle
car
hovercraft
boat

Inheritance in Java




Shape
TwoDimensionalShape
ThreeDimensionalShape
Circle
Square
Triangle
Tetrahedron
Cube
Sphere

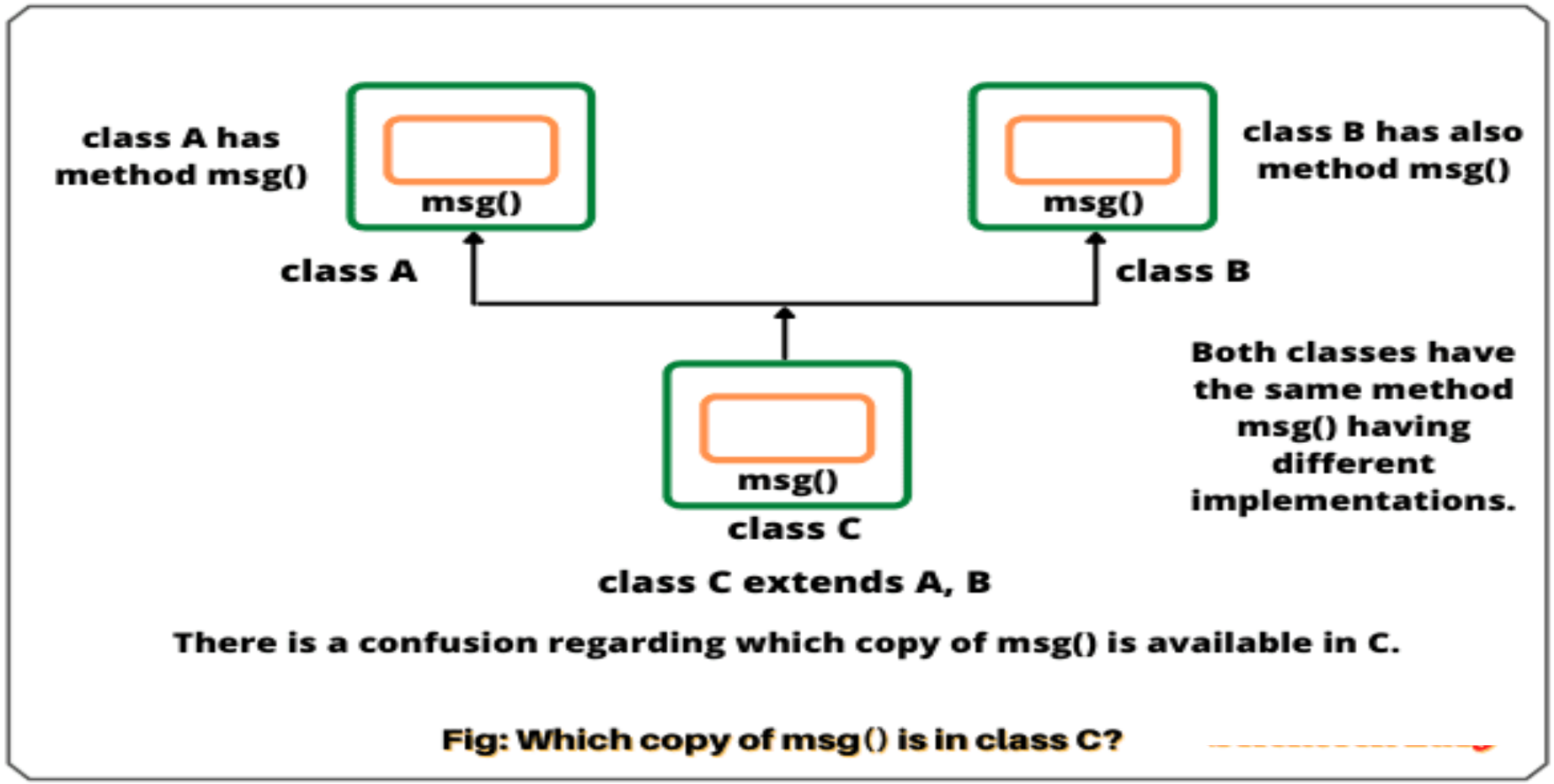
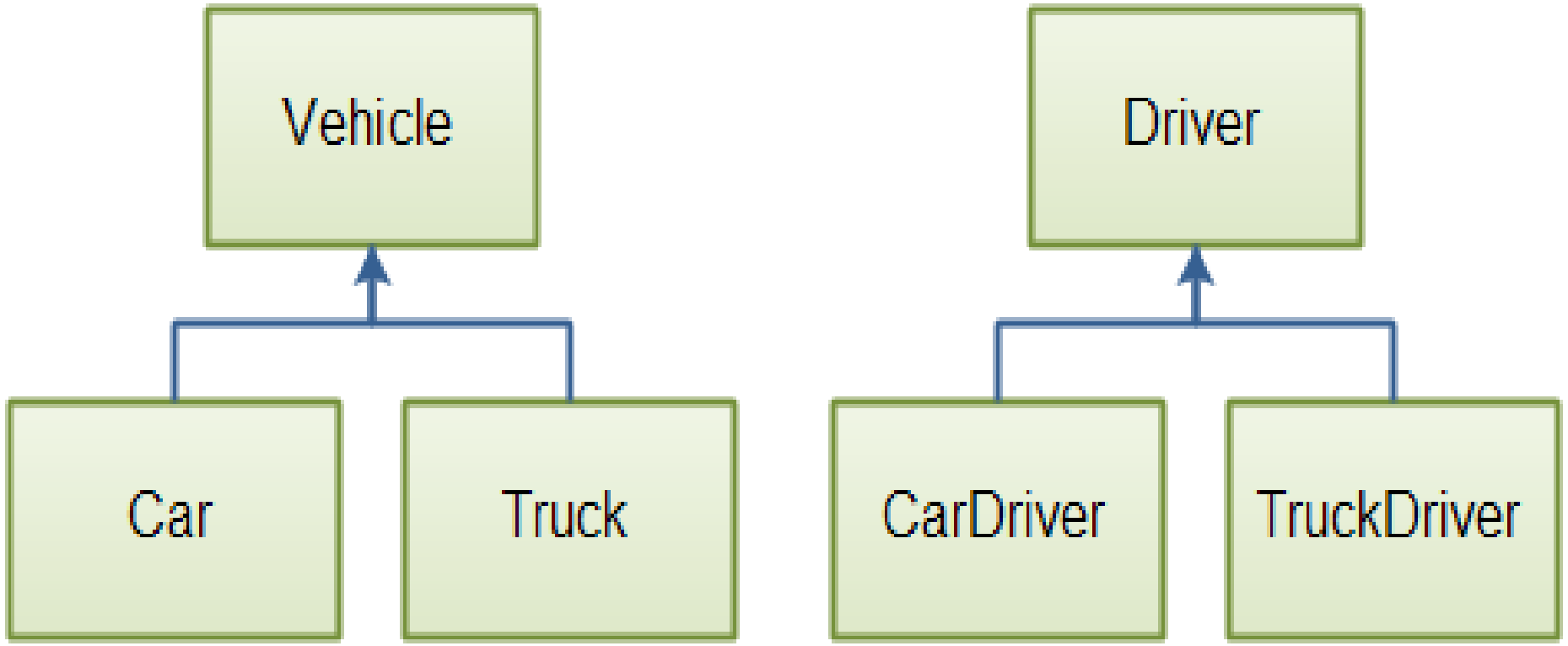
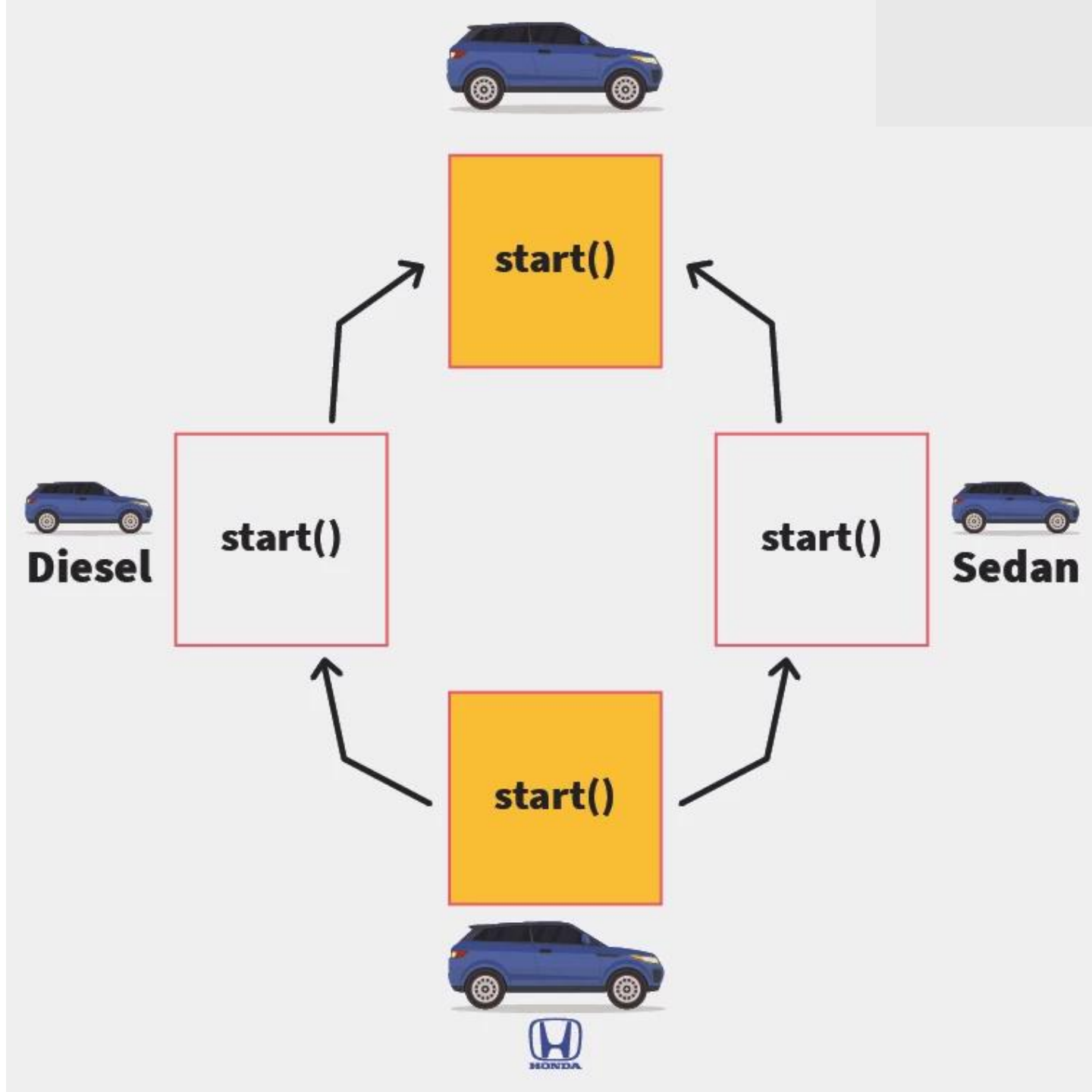


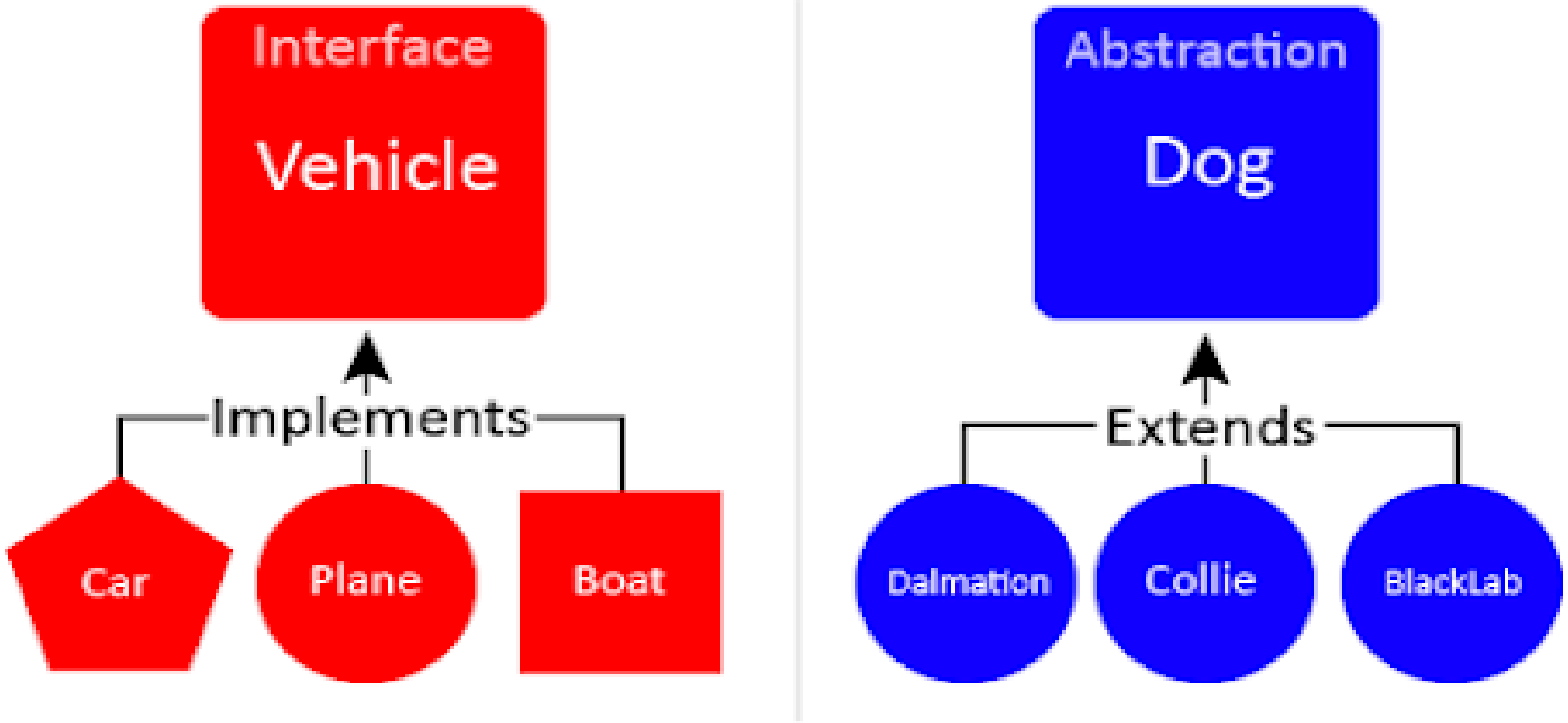
Fig: Which copy of msg() is in class C?

Interfaces
&
Abstract classes

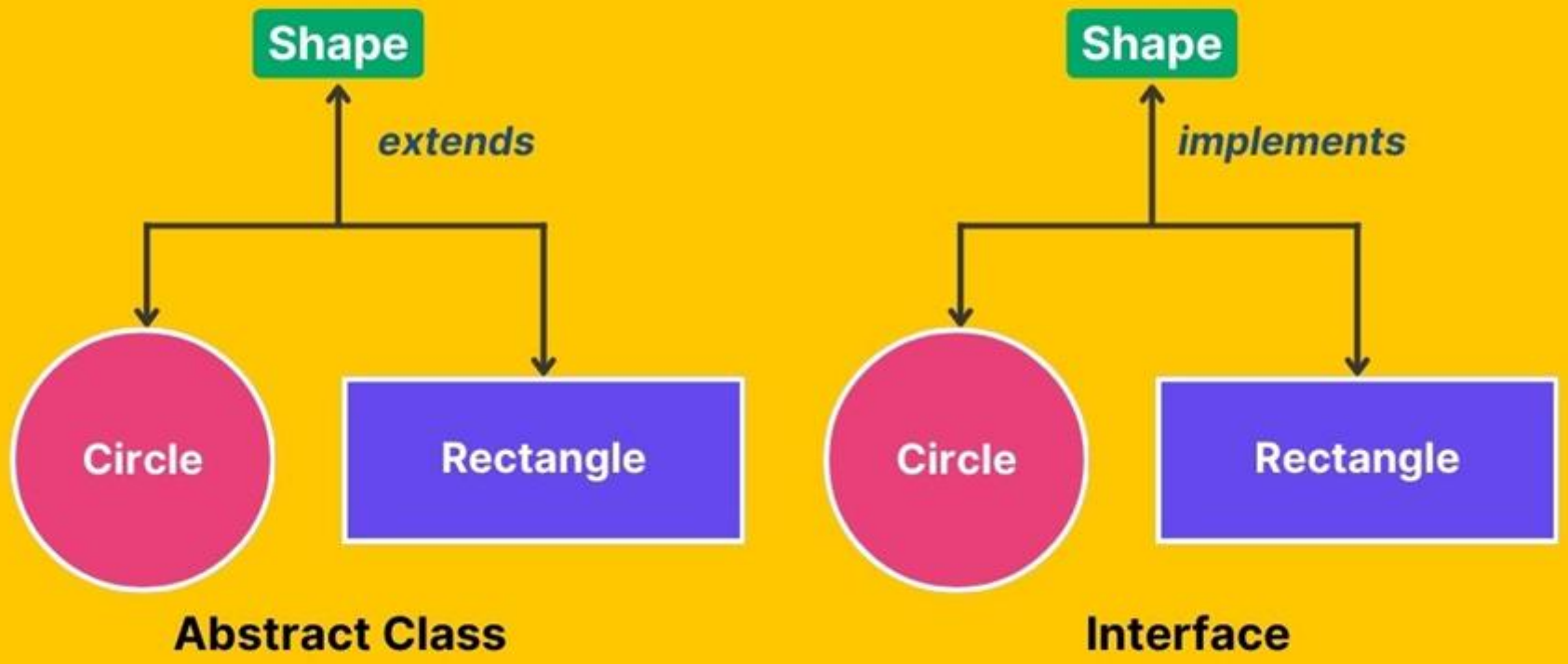

Vehicle
Car
Truck
Driver
CarDriver
TruckDriver

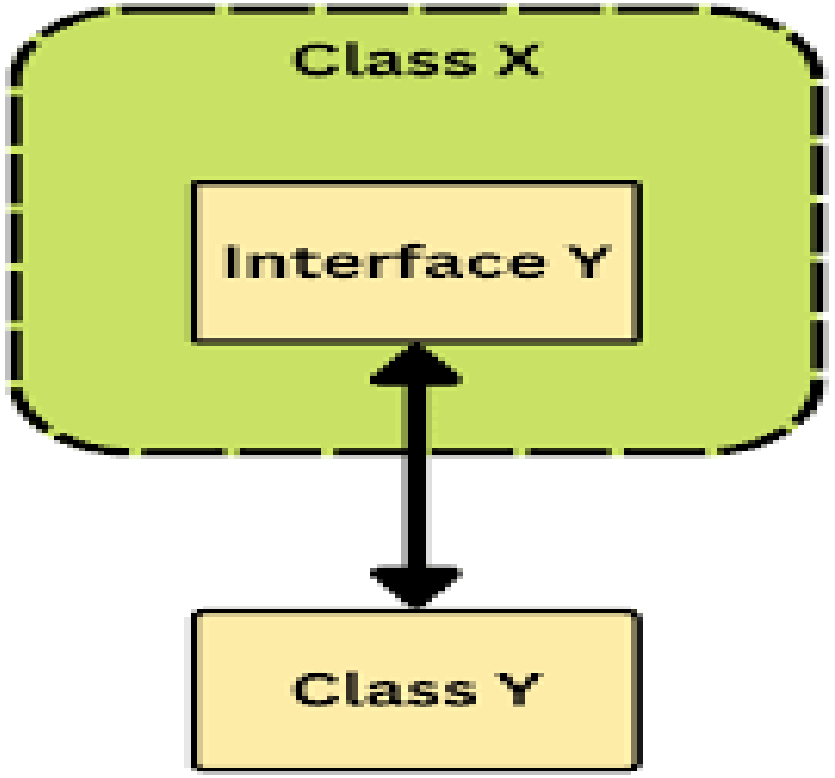

start()
start()
Diesel
start()
Sedan
start()
HONDA

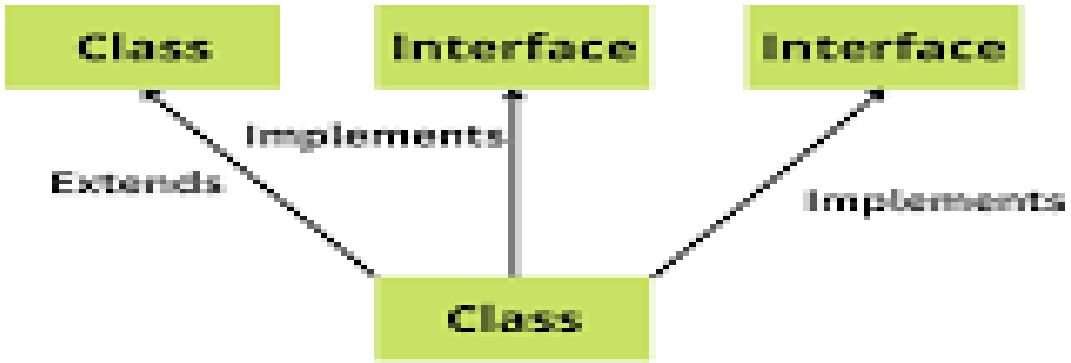
Interfaces vs. Abstract Classes

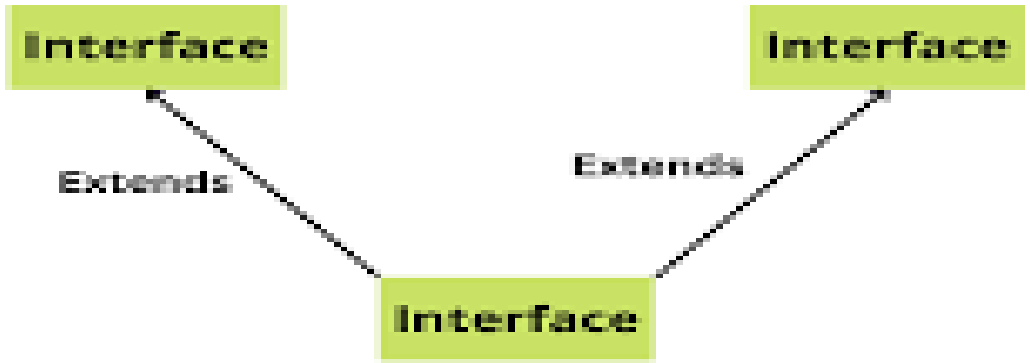


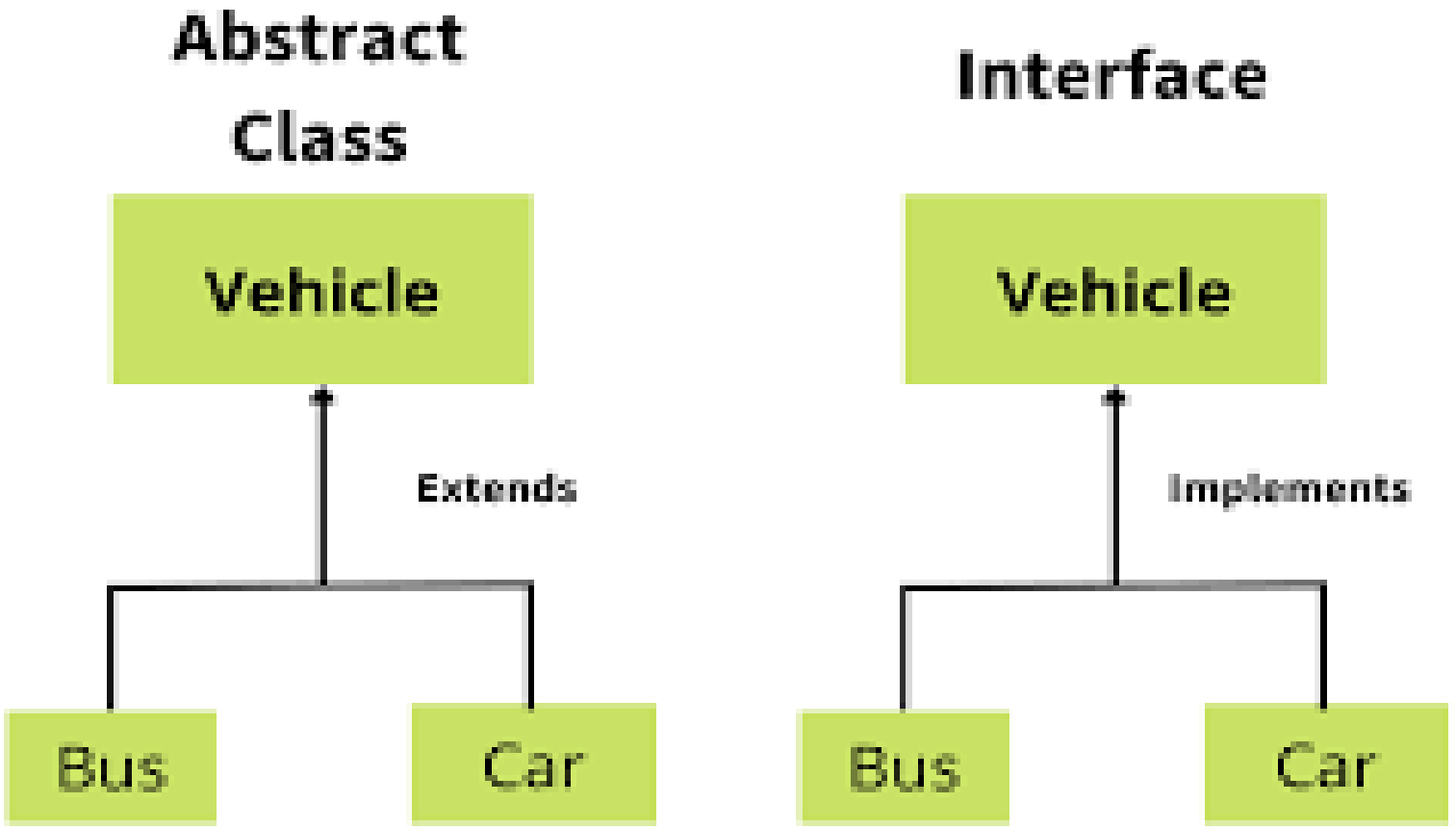
Abstract Class Vs. Interface In Java




Class X
Interface Y
Class Y


Class
Interface
Interface
Implements
Extends
Implements
Class


Interface
Interface
Extends
Extends
Interface


Abstract Class
Vehicle
Extends
Bus
Car
Interface
Vehicle
Implements
Bus
Car